

Cetmix Tower Server Management

Odoo based SAAS Devops

SRV 1
ODOO 18.0
ON-LINE

SRV 2
WORDPRESS
ON-LINE

SRV 3
ONLYOFFICE
ON-LINE

SRV 5
SHOPIFY
ON-LINE

SRV 4
ERP NEXT
OFF-LINE



Guía completa Cetmix Tower para efactusv

De la instalación a un SaaS Odoo en producción,
paso a paso

Alcance Configuración inicial · Servidores · Comandos · Flight Plans · Jets · Contrato de cliente · Instancia Odoo de producción · Dominio propio y SSL

Módulos cetmix_tower_server, cetmix_tower_yaml, cetmix_tower_git, cetmix_tower_webhook, cetmix_tower_ovh, cetmix_tower_aws, cetmix_tower_server_queue, cx_web_refresh_from_backend, efactusv_tower_contract

Versión Odoo 18.0 · Cetmix Tower 18.0.2.0.0 (fork efactusv)

Repositorio efactusv-cetmix-tower (rama 18.0)

Servidores
SSH

Comandos y Flight
Plans

Plantillas de archivo

Variables y secretos

Jets (instancias SaaS)

Máquina de estados

Contratos OCA

DNS OVH + Let's
Encrypt

Webhooks

YAML

Documento técnico interno. Las capturas provienen de la documentación oficial incluida en los módulos (static/description) y muestran datos de demostración.

Contenido

Guía completa Cetmix Tower para efactusv

Contenido	2
CAPÍTULO 1 Panorama: qué es Cetmix Tower y cómo encaja en efactusv	4
1.1 El modelo mental en siete piezas	4
1.2 Los módulos del repositorio y para qué sirve cada uno	4
1.3 Arquitectura de la plataforma SaaS que vamos a construir	5
1.4 Cómo leer esta guía	5
CAPÍTULO 2 Instalación y prerequisites	7
2.1 Dependencias de módulos Odoo	7
2.2 Dependencias Python	7
2.3 Orden de instalación de los módulos	7
2.4 Configuración de odoo.conf en el Odoo madre	8
2.5 Verificación de que todo quedó bien	9
CAPÍTULO 3 Configuración inicial	10
3.1 Grupos de seguridad y niveles de acceso	10
3.2 Ajustes generales	11
3.3 Sistemas operativos y etiquetas	11
3.4 Llaves SSH y secretos	12
3.5 Variables	12
CAPÍTULO 4 Tu primer servidor: del VPS vacío a la conexión verde	15
4.1 Preparar el VPS antes de tocar Odoo	15
4.2 Dar de alta el servidor en Tower	15
4.3 Probar la conexión	17
4.4 Anatomía de la ficha del servidor	17
4.5 Server Logs: ver la máquina sin abrir una terminal	17
CAPÍTULO 5 Comandos: la unidad de trabajo	19
5.1 Los seis tipos de acción	19
5.2 Campos comunes de un comando	19
5.3 Ejecutar un comando a mano	19
5.4 Comandos Python: el escape hacia el resto del mundo	20
5.5 Logs de comando	21
CAPÍTULO 6 Archivos y plantillas de archivo	22
6.1 Plantilla contra archivo	22
6.2 Crear una plantilla	22
6.3 Acciones sobre un archivo	23
6.4 Las tres plantillas de la instancia Odoo efactusv	23
CAPÍTULO 7 Flight Plans: procesos reproducibles	27
7.1 Anatomía de un plan	27
7.2 Las líneas del plan	27
7.3 Ejecutar un plan	28
7.4 Anidar planes	29
7.5 Códigos de salida de Tower	29
CAPÍTULO 8 Plantillas de servidor: alta de VPS repetible	31
8.1 Qué copia una plantilla	31
8.2 Crear un servidor desde la plantilla	31
8.3 Crear servidores desde código	32
CAPÍTULO 9 Jets: el motor SaaS	34
9.1 El vocabulario de los jets	34
9.2 Estados	34
9.3 Acciones: la máquina de estados	34
9.4 El Jet Template	35
9.5 Instalar la plantilla en un servidor	36
9.6 Crear y operar jets	36
9.7 Waypoints: puntos de restauración	37
9.8 Códigos de error del subsistema de jets	38
CAPÍTULO 10 Caso práctico: la plantilla «Instancia Odoo efactusv»	39
10.1 Inventario: lo que hay que tener antes de empezar	39
10.2 Paso 1 — Crear los estados	39
10.3 Paso 2 — Crear los flight plans	39

10.4 Paso 3 — Crear el Jet Template	41
10.5 Paso 4 — Instalar la plantilla en el servidor	42
10.6 Paso 5 — Lanzar la primera instancia	42
10.7 Asignación automática de puertos	43
CAPÍTULO 11 Dominio propio del cliente y SSL	45
11.1 Configurar el acceso a la API de OVHcloud	45
11.2 El subplan efactusv_instance_dns	45
11.3 El subplan efactusv_instance_ssl	46
11.4 El dominio propio del cliente	46
CAPÍTULO 12 Contratos: enlazar el negocio con la infraestructura	49
12.1 Qué añade el módulo	49
12.2 Configuración, una sola vez	49
12.3 El alta de un cliente, de principio a fin	49
12.4 Terminación y reactivación automáticas	50
12.5 Lo que este módulo deliberadamente no hace	50
CAPÍTULO 13 Addons y proyectos Git	52
13.1 Jerarquía de objetos	52
13.2 Montar el proyecto de addons de efactusv	52
13.3 Dos estrategias de despliegue de addons	53
13.4 Un plan de actualización seguro	53
CAPÍTULO 14 Automatización: webhooks, tareas programadas y cola	54
14.1 Ejecución asíncrona con cetmix_tower_server_queue	54
14.2 Tareas programadas	54
14.3 Webhooks entrantes	54
CAPÍTULO 15 YAML: versionar la configuración	57
15.1 Exportar	57
15.2 Importar	57
15.3 Un flujo de trabajo que funciona	57
CAPÍTULO 16 Operación diaria y diagnóstico	59
16.1 Rutinas	59
16.2 Diagnóstico por síntoma	59
16.3 Recuperar un jet atascado en estado transitorio	60
16.4 Qué vigilar en cada servidor	61
CAPÍTULO 17 Anexos	62
A. Mapa completo de menús	62
B. Referencia rápida de sintaxis	62
C. API de Python más usada	63
D. Lista de verificación: alta de un cliente nuevo	63
E. Lista de verificación: baja de un cliente	64
F. Glosario	64
G. Referencias	64

Panorama: qué es Cetmix Tower y cómo encaja en efactusv

Cetmix Tower convierte a Odoo en un panel de control de infraestructura. Desde la misma base de datos donde vive tu CRM, tus contratos y tu facturación, puedes conectarte por SSH a servidores, ejecutar comandos, subir archivos de configuración generados con plantillas, encadenar todo en procesos reproducibles y —con el sistema de *Jets*— modelar cada instancia Odoo de cliente como un objeto con ciclo de vida propio. Ese último punto es lo que lo convierte en una plataforma SaaS estilo Odoo.sh.

1.1 El modelo mental en siete piezas

Antes de tocar un solo botón conviene fijar el vocabulario. Todo lo demás en esta guía se construye sobre estos siete conceptos, y aparecen exactamente con estos nombres en la interfaz.

Concepto	Modelo técnico	Qué es
Server Servidor	<code>cx.tower.server</code>	Una máquina real (VPS, dedicado, VM) a la que Tower se conecta por SSH. Guarda IP, puerto, usuario, modo de autenticación, sistema operativo, etiquetas y su propio conjunto de variables.
Command Comando	<code>cx.tower.command</code>	Una unidad de trabajo reutilizable. Puede ser un comando SSH, código Python que corre en Tower, la creación de un archivo desde plantilla, la ejecución de un Flight Plan, el disparo de una acción de Jet o la creación de un waypoint.
Flight Plan Plan de vuelo	<code>cx.tower.plan</code>	Una secuencia ordenada de comandos con lógica condicional: qué hacer según el código de salida de cada paso, cuándo abortar, cuándo continuar. Es el equivalente a un playbook.
File / File Template Archivo y plantilla	<code>cx.tower.file</code> <code>cx.tower.file.template</code>	Archivos que Tower sube al servidor (o descarga de él). La plantilla contiene el cuerpo con variables <code>{{ }}</code> ; el archivo es la instancia concreta renderizada para un servidor o jet dado.
Variable	<code>cx.tower.variable</code> <code>cx.tower.variable.value</code>	Valores con nombre que se inyectan en comandos, rutas y archivos. Se resuelven en cascada: primero el jet, luego el servidor, luego el valor global.
Key / Secret Llave y secreto	<code>cx.tower.key</code>	Claves privadas SSH y secretos (contraseñas, tokens de API). Nunca se renderizan en los logs: aparecen enmascarados.
Jet	<code>cx.tower.jet</code> <code>cx.tower.jet.template</code>	Una <i>instancia de aplicación</i> que vive en un servidor. Para efactusv, un jet = la instancia Odoo de un cliente. Tiene estados (<code>running</code> , <code>stopped</code> ...), acciones que la mueven entre estados, dependencias y sus propios archivos y variables.

LA REGLA DE ORO

Servidor = el hierro. **Jet** = lo que corre encima y le facturas a un cliente. Un servidor puede alojar decenas de jets. El contrato de Odoo apunta a un jet, no a un servidor.

1.2 Los módulos del repositorio y para qué sirve cada uno

El repositorio `efactusv-cetmix-tower` contiene nueve módulos. No todos son obligatorios; la tabla indica cuáles necesitas para el flujo SaaS descrito en esta guía.

Módulo	¿Necesario?	Función
<code>cetmix_tower_server</code>	Obligatorio	El núcleo. Servidores, comandos, flight plans, archivos, variables, llaves, jets, tareas programadas, logs y toda la seguridad. Es el módulo grande (~17.000 líneas de Python).
<code>cx_web_refresh_from_backend</code>	Obligatorio	Dependencia técnica: permite que el backend refresque las vistas del navegador cuando un comando largo termina. Se instala solo, como dependencia.

Módulo	¿Necesario?	Función
<code>cetmix_tower</code>	Opcional	Módulo "paraguas": no aporta lógica, solo instala todos los demás de golpe. Útil para un entorno de pruebas completo.
<code>cetmix_tower_yaml</code>	Muy recomendado	Exporta e importa comandos, planes, plantillas y jet templates en YAML. Es tu mecanismo de control de versiones y de promoción entre entornos (dev → prod).
<code>cetmix_tower_server_queue</code>	Muy recomendado	Ejecuta comandos y flight plans de forma asíncrona vía <code>queue_job</code> (OCA). Sin él, un <code>docker compose pull</code> de cinco minutos bloquea el worker HTTP de Odoo.
<code>cetmix_tower_git</code>	Recomendado	Modela repositorios, remotos y "proyectos Git" (conjuntos de fuentes). Genera el <code>repos.yaml</code> de git-aggregator para el árbol de addons del servidor.
<code>cetmix_tower_webhook</code>	Opcional	Endpoints HTTP entrantes con autenticador propio. Permite que un formulario web, una pasarela de pago o un sistema externo dispare un aprovisionamiento.
<code>cetmix_tower_ovh</code>	Según proveedor	Añade la librería <code>ovh</code> al entorno de los comandos Python: crear registros DNS, listar y aprovisionar VPS en OVHcloud.
<code>cetmix_tower_aws</code>	Según proveedor	Lo mismo con <code>boto3</code> para AWS (EC2, Route 53, S3).

A ellos se suma un módulo que **no** está en este repositorio sino en `efactusv`:

Módulo	Repositorio	Función
<code>efactusv_tower_contract</code>	<code>efactusv</code>	El pegamento entre el negocio y la infraestructura. Añade al contrato OCA un selector de plantilla de instancia y de servidor, un botón Create Instance , y automatiza el ciclo de vida: al terminar el contrato la instancia se detiene; al cancelar la terminación, vuelve a arrancar. Se documenta en el capítulo 12.

1.3 Arquitectura de la plataforma SaaS que vamos a construir

El objetivo final de esta guía es el siguiente montaje. Cada capítulo construye una pieza.



1.4 Cómo leer esta guía

Los capítulos están ordenados como un camino: cada uno asume que el anterior está hecho. Si tu objetivo es llegar cuanto antes a un cliente en producción, este es el recorrido mínimo:

Cap.	Qué consigues	Puedes saltártelo si...
2	Tower instalado y arrancando	Nunca

Cap.	Qué consigues	Puedes saltártelo si...
3	Permisos, secretos y variables listos	Nunca
4	El primer servidor conectado por SSH	Nunca
5-7	Comandos, archivos y flight plans	Nunca: son los ladrillos del jet
8	Plantillas de servidor (alta de VPS repetible)	Solo tendrás uno o dos servidores
9-10	El motor SaaS: jets e instancia de cliente	Nunca, es el corazón
11	Subdominio, dominio propio del cliente y SSL	Nunca para producción
12	Contrato ↔ instancia automatizado	Facturarás a mano
13-15	Addons vía Git, automatización, YAML	En un primer piloto
16-17	Operación, diagnóstico y referencia	Guárdalos para el día 2

CONVENCIONES TIPOGRÁFICAS

`Menú > Submenú` indica una ruta de navegación en la interfaz de Odoo. **Botón** es un botón que hay que pulsar. `campo_tecnico` es un nombre de campo o modelo. Los bloques oscuros son código para copiar.

Instalación y prerequisites

Tower vive dentro de tu Odoo, así que "instalarlo" es tres cosas: poner los repositorios en `eaddons_path`, satisfacer las dependencias Python y de módulos OCA, y ajustar `odoo.conf` y `nginx` para que los trabajos largos y las notificaciones en vivo funcionen.

2.1 Dependencias de módulos Odoo

El núcleo declara `mail`, `rpc_helper`, `web_notify` y `cx_web_refresh_from_backend`. Los dos del medio vienen de repositorios OCA que deben estar clonados junto al fork de Tower:

Repositorio OCA	Módulo requerido	Lo necesita
server-tools	<code>rpc_helper</code>	<code>cetmix_tower_server</code>
web	<code>web_notify</code>	<code>cetmix_tower_server</code> (notificaciones en vivo)
queue	<code>queue_job</code>	<code>cetmix_tower_server_queue</code>
contract	<code>contract</code>	<code>efactusv_tower_contract</code>

2.2 Dependencias Python

El archivo `requirements.txt` del repositorio ya las agrega todas. Instálalas en el mismo entorno virtual donde corre Odoo:

```
cd /ruta/a/efactusv-cetmix-tower
pip install -r requirements.txt
```

Paquete	Lo usa	Para qué
<code>paramiko<4.0.0</code>	núcleo	Cliente SSH y SFTP. Es la pieza que ejecuta todo en el servidor remoto.
<code>tlsextract</code> , <code>dnspython</code>	núcleo	Validación de URLs y resolución de nombres de los servidores.
<code>ansi2html</code>	núcleo	Convierte la salida coloreada de los logs (códigos ANSI) en HTML legible en la vista de Server Logs.
<code>pyyaml</code>	<code>_yaml</code>	Serialización de la configuración.
<code>giturlparse==0.12.0</code>	<code>_git</code>	Análisis de URLs de repositorios (HTTPS/SSH/git).
<code>ovh</code>	<code>_ovh</code>	Cliente de la API de OVHcloud dentro de comandos Python.
<code>boto3</code>	<code>_aws</code>	SDK de AWS dentro de comandos Python.

NOTA SOBRE PYTHON 3.13

Python 3.13 eliminó `re.template`, que la versión original de Cetmix Tower usaba en `cx_tower_variable.py` y `cx_tower_command.py`. Este fork ya incorpora el arreglo (commit `c71707a`). Si en algún momento actualizas desde el upstream de Cetmix, verifica que el parche siga aplicado o la instalación fallará al importar los modelos.

2.3 Orden de instalación de los módulos

Instala desde **Aplicaciones** (con el filtro *Aplicaciones* desactivado, porque la mayoría son módulos técnicos) en este orden. Cada paso arrastra sus dependencias automáticamente.

1 cetmix_tower_server

Arrastra `mail`, `rpc_helper`, `web_notify` y `cx_web_refresh_from_backend`. Al terminar aparece el menú raíz **Cetmix Tower**.

2 cetmix_tower_yaml

Añade la pestaña **YAML** a comandos, planes, plantillas y jet templates, y el menú **Tools > Import YAML**. Instálalo pronto: querrás versionar la configuración desde el principio, no después de haberla construido a mano.

3 cetmix_tower_server_queue

Requiere `queue_job` ya configurado (ver 2.4). A partir de aquí los comandos y flight plans se pueden encolar en lugar de bloquear la petición HTTP.

4 cetmix_tower_git · cetmix_tower_webhook

Ambos dependen de `cetmix_tower_yaml`. Git añade **Tools > Git Projects**; Webhook añade **Settings > Automation > Webhooks** y **Webhook Authenticators**.

5 cetmix_tower_ovh y/o cetmix_tower_aws

Solo el de tu proveedor. No añaden interfaz: inyectan la librería correspondiente en el contexto de ejecución de los comandos Python.

6 efactusv_tower_contract

Último, porque necesita que existan tanto `contract` como `cetmix_tower_server`. Vive en el repositorio `efactusv`.

INSTALACIÓN DESDE LÍNEA DE COMANDOS

```
./odoo-bin -c odoo.conf -d efactusv_prod \  
-i cetmix_tower_server,cetmix_tower_yaml,cetmix_tower_server_queue,\  
cetmix_tower_git,cetmix_tower_webhook,cetmix_tower_ovh,efactusv_tower_contract \  
--stop-after-init
```

2.4 Configuración de `odoo.conf` en el Odoo madre

Dos ajustes son imprescindibles y uno es muy recomendable.

queue_job cargado a nivel de servidor

`queue_job` necesita registrarse como *server-wide module*; no basta con instalarlo en la base de datos. Además, la sección `[queue_job]` debe ir **después** del bloque `[options]`, o el parser de Odoo la ignora silenciosamente.

```
[options]  
addons_path = /opt/odoo/odoo/addons,/opt/odoo/efactusv,/opt/odoo/efactusv-cetmix-tower,\  
/opt/odoo/queue,/opt/odoo/server-tools,/opt/odoo/web,/opt/odoo/contract  
server_wide_modules = base,web,queue_job  
workers = 4  
max_cron_threads = 2  
limit_time_real = 1200  
limit_time_cpu = 600  
  
[queue_job]  
channels = root:2
```

WORKERS > 0 ES OBLIGATORIO

Con `workers = 0` (modo hilo único) `queue_job` no procesa nada y los comandos largos de Tower bloquean toda la instancia. En producción usa siempre modo multiproceso.

nginx: proxy del websocket

Tower notifica en tiempo real el fin de cada comando mediante el bus de Odoo 18, que usa websockets sobre el puerto `gevent` (8072). Si nginx no lo proxee, verás los comandos "colgados" en la interfaz aunque hayan terminado bien en el servidor.

```
location /websocket {
    proxy_pass http://127.0.0.1:8072;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header X-Forwarded-Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Real-IP $remote_addr;
}
```

Tiempos de espera generosos

Un `docker compose pull` de una imagen Odoo puede tardar varios minutos. Sube `limit_time_real` a 1200 s y, en nginx, `proxy_read_timeout 720s` como mínimo. Si además usas `cetmix_tower_server_queue` el riesgo baja mucho, porque el trabajo pesado sale de la petición HTTP.

2.5 Verificación de que todo quedó bien

Comprobación	Resultado esperado
Aparece el menú Cetmix Tower en la barra superior	Con submenús Servers, Jets, Logs, Commands, Files, Tools, Settings
<code>Cetmix Tower > Settings > General Settings</code> abre	Se ven los campos de timeout y notificaciones
<code>python -c "import paramiko, ansi2html, tldextract"</code>	Sin errores
Log de Odoo al arrancar	Contiene <code>Starting job runner</code> (<code>queue_job</code> activo)
Consola del navegador en una vista de Tower	Websocket conectado, sin errores 404 en <code>/websocket</code>

Configuración inicial

Antes de conectar un servidor hay que preparar el terreno: quién puede hacer qué, cómo se guardan las credenciales y qué valores parametrizables va a usar toda la plataforma. Saltarse este capítulo es la causa número uno de "no veo el botón" y de secretos filtrados en logs.

3.1 Grupos de seguridad y niveles de acceso

Tower usa **dos mecanismos superpuestos**. Entender la diferencia ahorra horas de depuración.

Mecanismo 1: grupos de Odoo

Definen qué menús y qué modelos ve un usuario. Son jerárquicos: Root implica Manager, que implica User.

Grupo	XML ID	Qué puede hacer
User	group_user	Ver los servidores y jets a los que ha sido asignado, ejecutar comandos y flight plans marcados con nivel de acceso "User", consultar logs.
Manager	group_manager	Todo lo anterior, más crear servidores, plantillas de servidor, comandos, flight plans, variables, tareas programadas y jet templates.
Root	group_root	Todo. Es el único que puede configurar webhooks y autenticadores, y ver secretos de nivel Root. Resérvalo para dos o tres personas.

Mecanismo 2: Access Level en el usuario y en cada registro

Casi todos los objetos de Tower (comando, plan, archivo, plantilla, variable, secreto, estado de jet, línea de plan...) tienen un campo `access_level` con tres valores: **1 = User**, **2 = Manager** (el valor por defecto) y **3 = Root**. El usuario tiene su propio nivel en su ficha. Un usuario solo ve y ejecuta registros cuyo nivel sea **menor o igual** al suyo.

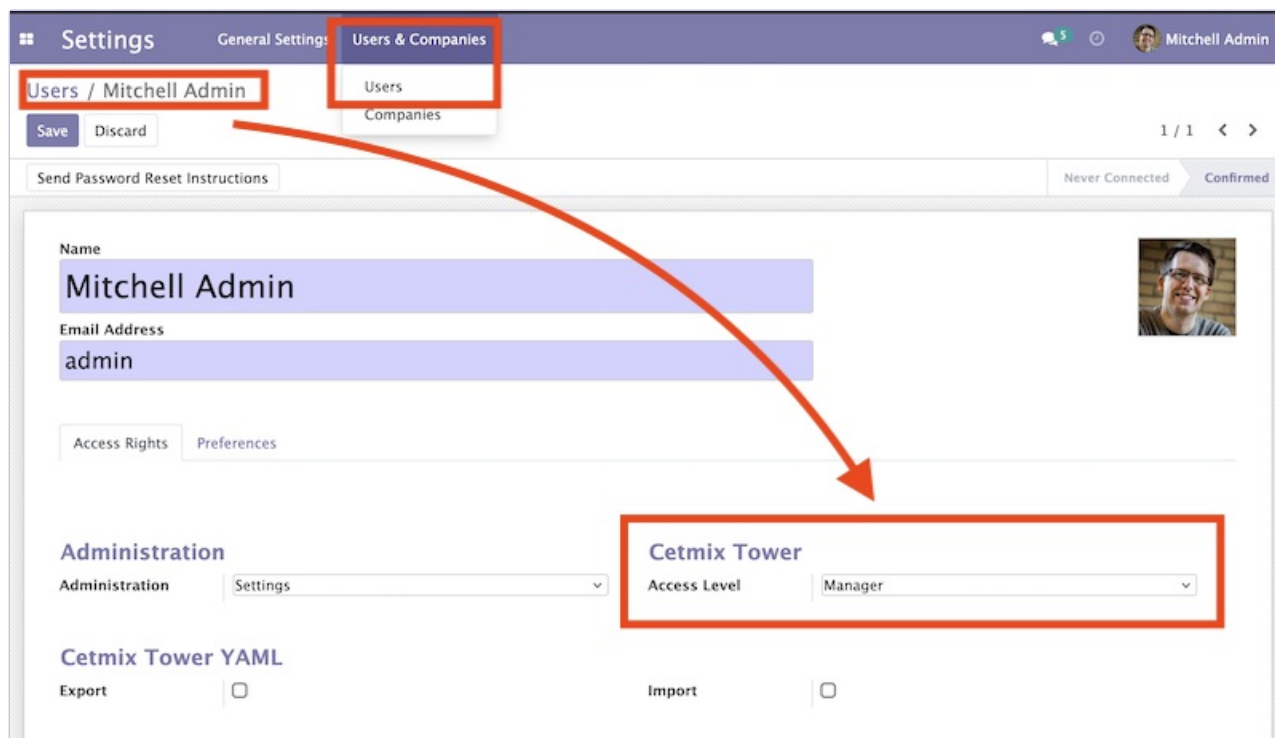


Figura 3.1 — El nivel de acceso de Tower se fija en la ficha del usuario, en `Ajustes > Usuarios y compañías > Usuarios`, pestaña Permisos de acceso, sección Cetmix Tower. Justo debajo aparecen las casillas de exportación/importación YAML.

EL ERROR CLÁSICO

Un usuario está en el grupo Manager pero su *Access Level* es "User": verá los menús pero la lista de comandos aparecerá casi vacía, porque los comandos se crean por defecto con nivel Manager. Ambos ajustes tienen que ir de la mano.

Mecanismo 3: listas de acceso por registro

Servidores, jets, plantillas, comandos, planes y llaves tienen además dos campos de relación: **Users** (quién puede usarlo) y **Managers** (quién puede modificarlo), en la pestaña **Access**. Esto permite, por ejemplo, dar a un técnico acceso solo a los servidores de un cliente concreto.

Plan de permisos recomendado para efactusv

Perfil	Grupo	Access Level	Alcance
Arquitecto de plataforma	Root	Root	Todo: webhooks, secretos de infraestructura, jet templates
DevOps / soporte N2	Manager	Manager	Servidores y jets asignados, crear comandos y planes
Soporte N1	User	User	Reiniciar y detener instancias vía acciones de jet, leer logs
Comercial / administración	—	—	Solo el contrato en Odoo; nunca entra a Tower

3.2 Ajustes generales

Ve a `Cetmix Tower > Settings > General Settings`.

Ajuste	Sugerido	Explicación
Command Timeout	900 s	Segundos tras los cuales un comando SSH se da por muerto y se termina con el código <code>-206</code> . Súbelo si aprovisionas instancias con <code>docker pull</code> de imágenes grandes; bájalo si prefieres detectar cuelgues rápido.
Error Notifications	Sticky	Las notificaciones de error se quedan en pantalla hasta que el usuario las cierra. Con <i>Non-sticky</i> desaparecen solas y es fácil perderse un fallo.
Success Notifications	Non-sticky	Los éxitos no necesitan interrumpir; que se desvanezcan.

En la misma pantalla hay tres botones que abren directamente las acciones planificadas de Odoo. Revisa sus intervalos:

Cron	Intervalo	Qué hace
Auto pull files from server	1 hora	Descarga los archivos marcados con <i>Auto Sync</i> según su propio intervalo.
Check zombie commands	15 min	Marca como fallidos los comandos que superaron el timeout y quedaron colgados.
Run scheduled tasks	5 min	Dispara las tareas programadas de Tower (backups, mantenimiento). Es el reloj del sistema: si va lento, tus backups van lentos.

Si instalaste `cetmix_tower_webhook`, aparece también **Keep Webhook Logs for (days)**, con 30 por defecto.

3.3 Sistemas operativos y etiquetas

En `Settings > Properties > Operating Systems` defines los SO que manejas (*Debian 12, Ubuntu 24.04...*). No es decorativo: cada comando puede declarar en qué SO es válido, y Tower **se niega a ejecutarlo** en un servidor incompatible devolviendo el código `-207`. Es una red de seguridad barata contra ejecutar `unapt` en un CentOS.

En `Settings > Properties > Tags` creas etiquetas de color para clasificar servidores, comandos y planes. Un esquema que funciona bien:

Etiqueta	Uso
<code>producción</code>	Servidores y planes que tocan clientes reales
<code>staging</code>	Entorno de pruebas

Etiqueta	Uso
mantenimiento	Comandos seguros de diagnóstico (df, docker ps, logs)
destrutivo	Comandos que borran datos: exige revisión antes de ejecutar

3.4 Llaves SSH y secretos

Ve a **Cetmix Tower > Settings > Keys and Secrets**. Hay dos tipos de registro, distinguidos por el campo **Key Type**.

Tipo "SSH Key" — la llave privada de conexión

1 Genera el par de llaves en tu máquina

```
ssh-keygen -t ed25519 -C "cetmix-tower@efactusv" -f ~/.ssh/tower_efactusv
```

No le pongas passphrase: Tower no puede introducirla de forma interactiva.

2 Crea el registro en Tower

Pulsa **Nuevo**. Rellena:

- **Name:** Tower efactusv - producción
- **Reference:** déjalo vacío y se genera solo `tower_efactusv_produccion`), o escribe uno corto y estable como `ssh_prod`. La referencia es lo que usarás desde la API.
- **Key Type:** *SSH Key*
- **Access Level:** *Root* — es la llave maestra de tu infraestructura.
- En la pestaña de valor, pega el contenido íntegro de `~/.ssh/tower_efactusv` (la clave **privada**, incluidas las líneas `-----BEGIN...` y `-----END...`).

3 Autoriza la clave pública en el servidor

```
ssh-copy-id -i ~/.ssh/tower_efactusv.pub deploy@203.0.113.10
```

Tipo "Secret" — contraseñas y tokens

Un secreto se referencia en cualquier comando, plantilla o archivo con esta sintaxis:

```
#!cxtower.secret.<referencia>!#
```

Ejemplo real: el secreto con referencia `ovh_application_key` se escribe `#!cxtower.secret.ovh_application_key!#`. Tower lo sustituye por el valor justo antes de ejecutar, y en los logs guarda el marcador enmascarado en lugar del valor.

POR QUÉ ESTO IMPORTA

Las variables normales `{ { }}` **sí** aparecen en los logs de comando. Los secretos `#!cxtower.secret.x!#` **no**. Cualquier contraseña de base de datos, `admin_passwd` o token de API debe ir como secreto, nunca como variable.

Secretos que necesitarás en esta guía

Referencia	Nivel	Contenido
<code>db_password</code>	Root	Contraseña del usuario PostgreSQL de las instancias de cliente
<code>admin_passwd</code>	Root	Contraseña maestra de Odoo de las instancias
<code>ovh_application_key</code>	Root	Credencial de la API de OVHcloud
<code>ovh_application_secret</code>	Root	Credencial de la API de OVHcloud
<code>ovh_consumer_key</code>	Root	Credencial de la API de OVHcloud
<code>ghcr_token</code>	Root	Token de lectura del registro de contenedores donde publicas la imagen efactusv

3.5 Variables

Ve a `Cetmix Tower > Settings > Variables > Variables`. Una variable es un nombre; su valor se define en varios niveles y Tower resuelve el más específico.

La cascada de resolución

```
Valor en el JET           (el más específico – gana siempre)
  ↳ si no existe
Valor en el SERVIDOR
  ↳ si no existe
Valor GLOBAL de la variable
  ↳ si no existe
(vacío, o error si la variable está marcada como requerida)
```

Esto es exactamente lo que hace que un mismo Flight Plan sirva para cien clientes: el plan dice`{{ instance_name }}` y cada jet aporta el suyo.

Tipos de variable

Tipo	Descripción
String	Texto libre. Puede llevar un patrón de validación (expresión regular) y un mensaje de error propio. Úsalo, por ejemplo, para forzar que <code>instance_name</code> sea <code>^[a-z][a-z0-9_]{2,20}\$</code> .
Options	Lista cerrada de opciones (<code>cx.tower.variable.option</code>), cada una con su etiqueta visible y su valor real. Ideal para <code>odoo_version</code> o para el plan contratado.

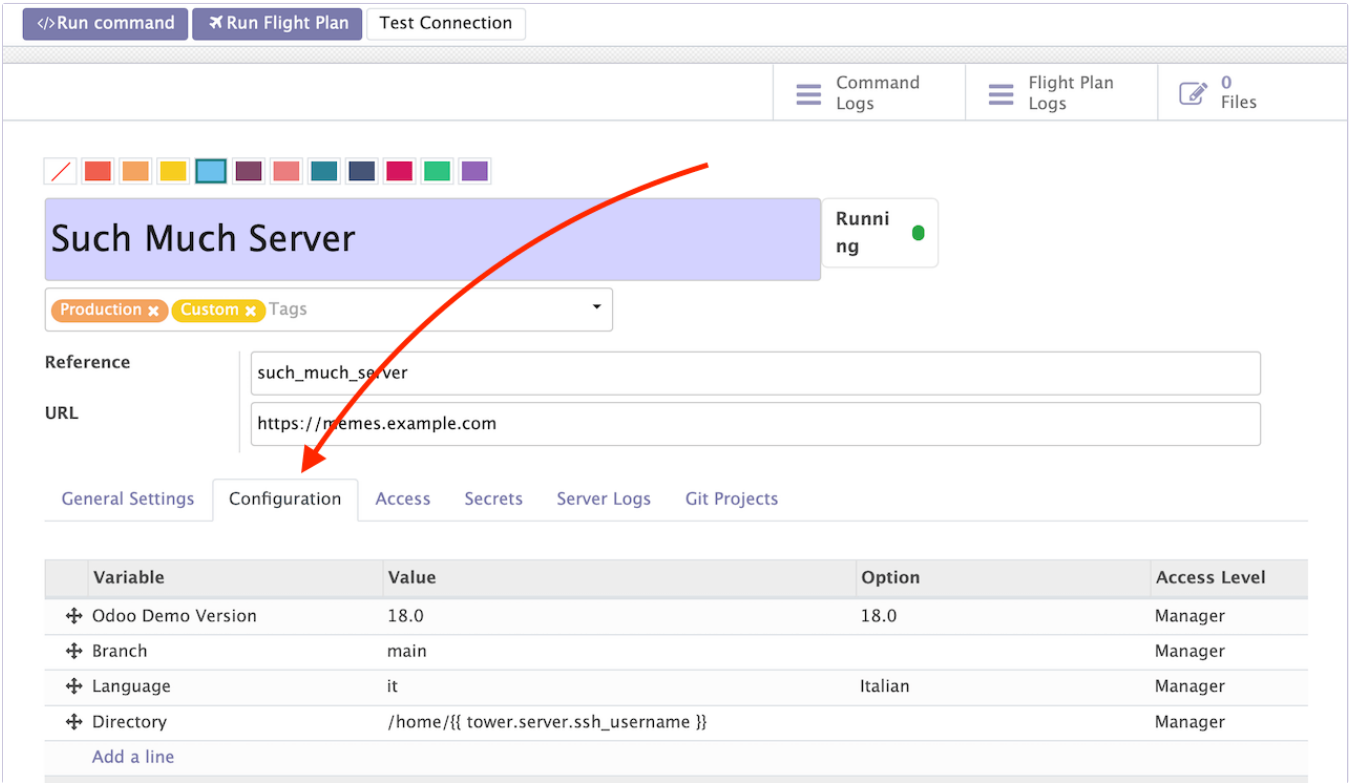


Figura 3.2 — Pestaña Configuration de un servidor: aquí se sobrescriben las variables para esa máquina concreta. Fíjate en la última fila: el valor de una variable puede a su vez contener otra variable (`/home/{{ tower.server.ssh_username }}`), y en la columna Access Level, que controla quién puede verla y modificarla.

Variables del sistema: el espacio de nombres tower

Además de las tuyas, Tower expone un conjunto de variables de solo lectura bajo el prefijo `tower.`. Se rellenan en tiempo de ejecución según el contexto (servidor, jet, plantilla). No hay que declararlas: siempre están disponibles.

Variable	Valor
<code>{{ tower.server.name }}</code>	Nombre del servidor
<code>{{ tower.server.reference }}</code>	Referencia técnica del servidor
<code>{{ tower.server.ipv4 }}</code> / <code>.ipv6</code>	Direcciones IP
<code>{{ tower.server.username }}</code>	Usuario SSH configurado

Variable	Valor
<code>{{ tower.server.status }}</code>	Estado: <code>running</code> , <code>stopped</code> ...
<code>{{ tower.server.os }}</code>	Sistema operativo
<code>{{ tower.server.url }}</code> · <code>.hostname</code> · <code>.netloc</code> · <code>.port</code>	URL del servidor y sus partes ya descompuestas
<code>{{ tower.server.partner_name }}</code>	Cliente asociado al servidor
<code>{{ tower.jet.name }}</code> · <code>.reference</code> · <code>.state</code>	Datos del jet en ejecución
<code>{{ tower.jet.url }}</code> · <code>.hostname</code> · <code>.netloc</code> · <code>.port</code>	URL del jet y sus partes
<code>{{ tower.jet.cloned_from }}</code>	Referencia del jet origen si este es un clon
<code>{{ tower.jet.waypoint.reference }}</code> · <code>.type</code>	Waypoint actual del jet y su tipo
<code>{{ tower.jet_template.name }}</code> · <code>.reference</code>	Plantilla de la que nació el jet
<code>{{ tower.tools.uuid }}</code>	UUID aleatorio nuevo en cada renderizado
<code>{{ tower.tools.today }}</code> · <code>.now</code>	Fecha y fecha-hora actuales
<code>{{ tower.tools.today_underscore }}</code> · <code>.now_underscore</code>	Igual, pero con guiones bajos: aptas para nombres de archivo

EJEMPLO DE USO COMBINADO

```
docker exec db-{{ instance_name }} pg_dump -U odoo -Fc {{ instance_name }} \
> /opt/efactusv/backups/{{ instance_name }}-{{ tower.tools.today_underscore }}.dump
```

Una sola línea que produce un nombre de archivo distinto y ordenable para cada cliente y cada día, sin escribir una sola fecha a mano.

Variables a crear ahora para el proyecto efactusv

Referencia	Tipo	Ejemplo	Uso
<code>instance_name</code>	String	<code>cliente1</code>	Identificador técnico: rutas, contenedores, nombre de la BD. Minúsculas, sin espacios.
<code>instance_domain</code>	String	<code>cliente1.efactusv.com</code>	Subdominio público de la instancia
<code>odoo_image</code>	String	<code>ghcr.io/porti1876/efactusv-odoo:18.0</code>	Imagen Docker a desplegar
<code>odoo_http_port</code>	String	<code>10069</code>	Puerto local único por instancia
<code>odoo_gevent_port</code>	String	<code>10072</code>	Puerto del websocket, único por instancia
<code>odoo_workers</code>	Options	<code>2 / 4 / 8</code>	Según el plan contratado
<code>base_dir</code>	String	<code>/opt/efactusv</code>	Raíz de despliegue. Global: igual en todos los servidores.
<code>admin_email</code>	String	<code>admin@efactusv.com</code>	Correo para Let's Encrypt

Tu primer servidor: del VPS vacío a la conexión verde

Este capítulo cubre las dos mitades del problema: preparar la máquina Linux para que sea un buen anfitrión de instancias Odoo, y darla de alta en Tower de forma que la prueba de conexión pase a la primera.

4.1 Preparar el VPS antes de tocar Odoo

Tower no aprovisiona el sistema base; espera encontrar una máquina con SSH, Docker y nginx. Estos son los requisitos mínimos para el modelo de esta guía.

Requisito	Detalle
Sistema operativo	Debian 12 o Ubuntu 24.04 LTS
Recursos	Regla práctica: 2 vCPU y 4 GB por cada 2-3 instancias pequeñas. El disco es lo que primero se agota: SSD de 80 GB en adelante.
Usuario	Un usuario no-root (p. ej. <code>deploy</code>) con <code>sudo</code> sin contraseña
Docker	Docker Engine + plugin <code>compose</code> ; el usuario debe estar en el grupo <code>docker</code>
nginx	Instalado en el host , no en contenedor: es quien enruta los subdominios a los puertos de cada instancia
certbot	Con el plugin de nginx, para los certificados de cada dominio
Cortafuegos	22, 80 y 443 abiertos. Los puertos de las instancias (10069, 10169...) escuchan solo en <code>127.0.0.1</code> , nunca se exponen.

Guion de preparación

```
# --- como root, una sola vez por servidor ---
adduser --disabled-password --gecos "" deploy
usermod -aG sudo deploy
echo "deploy ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/deploy

# Docker
curl -fsSL https://get.docker.com | sh
usermod -aG docker deploy

# nginx y certbot en el host
apt-get update
apt-get install -y nginx certbot python3-certbot-nginx
mkdir -p /var/www/certbot

# Raíz de despliegue de las instancias
mkdir -p /opt/efactusv/backups
chown -R deploy:deploy /opt/efactusv

# Autoriza la clave pública de Tower
mkdir -p /home/deploy/.ssh
cat >> /home/deploy/.ssh/authorized_keys <<'EOF'
ssh-ed25519 AAAA... cetmix-tower@efactusv
EOF
chown -R deploy:deploy /home/deploy/.ssh
chmod 700 /home/deploy/.ssh && chmod 600 /home/deploy/.ssh/authorized_keys
```

¿SUDO CON O SIN CONTRASEÑA?

El campo **Use sudo** del servidor ofrece *Without password* y *With password*. La segunda opción exige guardar la contraseña del usuario en Tower. Es más limpio configurar `NOPASSWD` para el usuario de despliegue y elegir *Without password*: así no hay una contraseña de sudo circulando por el sistema.

4.2 Dar de alta el servidor en Tower

Ve a `Cetmix Tower > Servers > Servers` y pulsa **Nuevo**. La vista por defecto es un kanban con el estado de cada máquina.

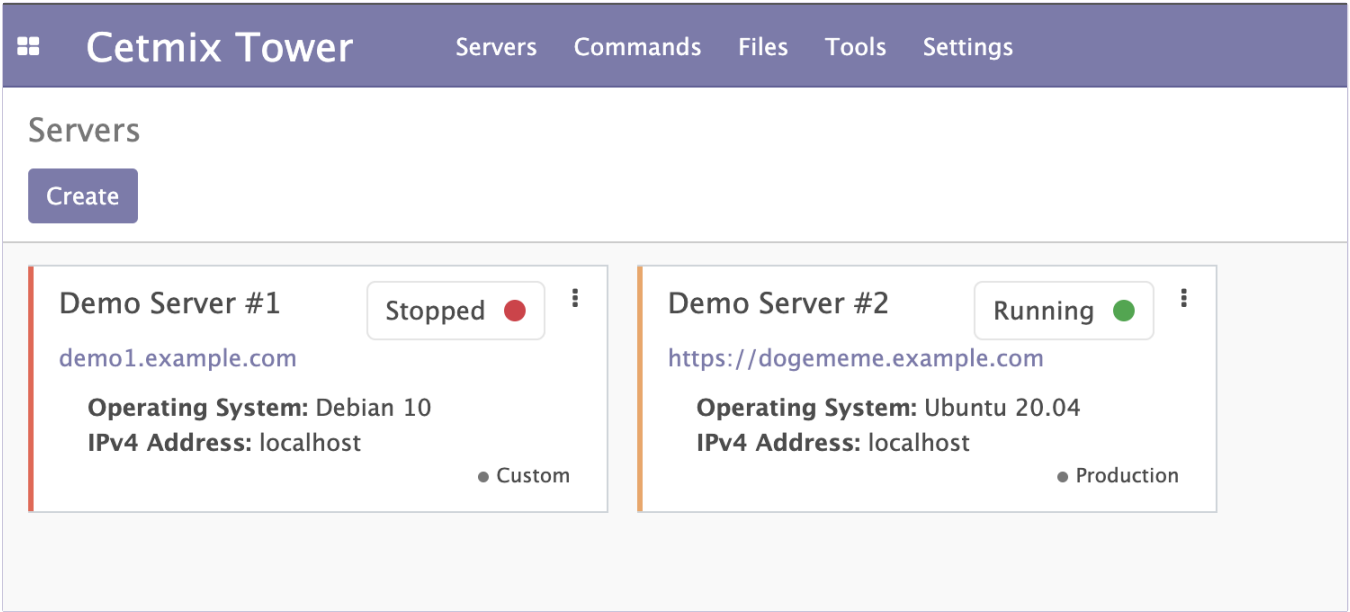


Figura 4.1 — Vista kanban de servidores. Cada tarjeta muestra la URL, el sistema operativo, la IPv4 y las etiquetas; el badge de la derecha es el campo status, con su semáforo.

Campos de la ficha, uno a uno

Campo	Obligatorio	Qué poner
Name	Sí	Nombre legible: <code>PROD-SV-01</code>
Reference	Auto	Déjalo vacío para autogenerar. Es el identificador estable que usarás desde código y desde YAML; una vez fijado, no lo cambies.
URL	No	Panel web del servidor si lo tiene. Alimenta <code>{{ tower.server.url }}</code> y sus derivados <code>.hostname</code> , <code>.port</code> .
Partner	No	Cliente propietario, si el servidor es dedicado a uno solo. En un servidor multiinquilino, déjalo vacío.
Operating System	Recomendado	El que creaste en 3.3. Habilita la validación de compatibilidad de comandos.
Tags	Recomendado	<code>producción</code> , la región, el proveedor...
IPv4 Address / IPv6 Address	Una de las dos	La dirección a la que Tower abre el SSH
SSH port	Sí	22 por defecto
SSH Username	Sí	<code>deploy</code>
SSH Auth Mode	Sí	Key en producción. Password solo para pruebas rápidas.
SSH Private Key	Si modo Key	La llave del capítulo 3. El desplegable solo muestra registros de tipo SSH Key.
Use sudo	No	Without password. Vacío si el usuario ya puede hacer todo sin sudo.
Host key	Ver abajo	Huella del servidor, para prevenir ataques de intermediario

La verificación de host key

Tower **se niega a conectar** si el servidor tiene IP pero no tiene `host_key` y la casilla **Skip host key** está desmarcada: lanza el error `"Host key not found for server X"`. Hay dos caminos:

A Recomendado: obtener y fijar la huella

Guarda el servidor y usa la acción **Show host key**, que se conecta una vez, lee la huella y la guarda en el campo. A partir de ahí, si la huella cambia (reinstalación del servidor, o un ataque), la conexión falla y te enteras.

B Atajo: marcar *Skip host key*

Válido en un laboratorio o cuando el servidor se reconstruye constantemente. **No lo uses en producción:** desactivas la única defensa contra un man-in-the-middle en el canal por el que envías tus contraseñas de base de datos.

4.3 Probar la conexión

Con el servidor guardado aparecen tres botones en la cabecera: **Run command**, **Run Flight Plan** y **Test Connection**.

Pulsa **Test Connection**. La prueba hace tres cosas en secuencia, y por eso es un diagnóstico tan útil:

1. **Abre la sesión SSH** con las credenciales y la huella configuradas.
2. **Ejecuta un comando de prueba** y comprueba que devuelve salida.
3. **Prueba operaciones de archivo:** sube un archivo temporal por SFTP, lo lee y lo borra.

El resultado llega como notificación. Si algo falla, el mensaje indica en qué fase.

Síntoma	Causa habitual
<i>Host key not found for server...</i>	Falta ejecutar Show host key o marcar <i>Skip host key</i>
<i>Authentication failed</i>	La clave pública no está en <code>authorized_keys</code> , o el SSH Auth Mode dice <i>Password</i> mientras pusiste una llave
Conecta pero falla el paso de archivos	El usuario no tiene permiso de escritura en su <code>\$HOME</code> , o el servidor no tiene subsistema SFTP habilitado
Se queda esperando y expira	Cortafuegos del proveedor, o el puerto SSH no es el 22
<i>Permission denied</i> al usar <code>sudo</code>	<code>NOPASSWD</code> no está configurado y Use sudo está en <i>Without password</i>

4.4 Anatomía de la ficha del servidor

Las pestañas del formulario organizan todo lo que Tower sabe de esa máquina:

Pestaña	Contenido
General Settings	Conexión, SO, etiquetas, partner, plan de borrado
Configuration	Valores de variables específicos de este servidor (Figura 3.2)
Access	Usuarios y managers con acceso a esta máquina
Secrets	Secretos con valor propio para este servidor. Permite que <code>db_password</code> sea distinto en cada VPS usando la misma referencia en los comandos.
Server Logs	Ventanas de log configuradas: ver 4.5
Git Projects	Proyectos de código asociados (capítulo 13)

Arriba a la derecha, los botones inteligentes llevan a **Command Logs**, **Flight Plan Logs**, **Files**, **Jets** y **Jet Templates** del servidor.

4.5 Server Logs: ver la máquina sin abrir una terminal

En la pestaña **Server Logs** defines "ventanas" de observación que cualquier usuario con acceso puede abrir con un clic, sin credenciales SSH propias. Cada entrada tiene un **Log Type**:

Tipo	Qué muestra
File	El contenido de un archivo del servidor (un <code>*.log</code>), a través de un registro de tipo archivo
Command	La salida de un comando ejecutado en el momento: <code>docker logs</code> , <code>df -h</code> , <code>docker stats --no-stream</code>

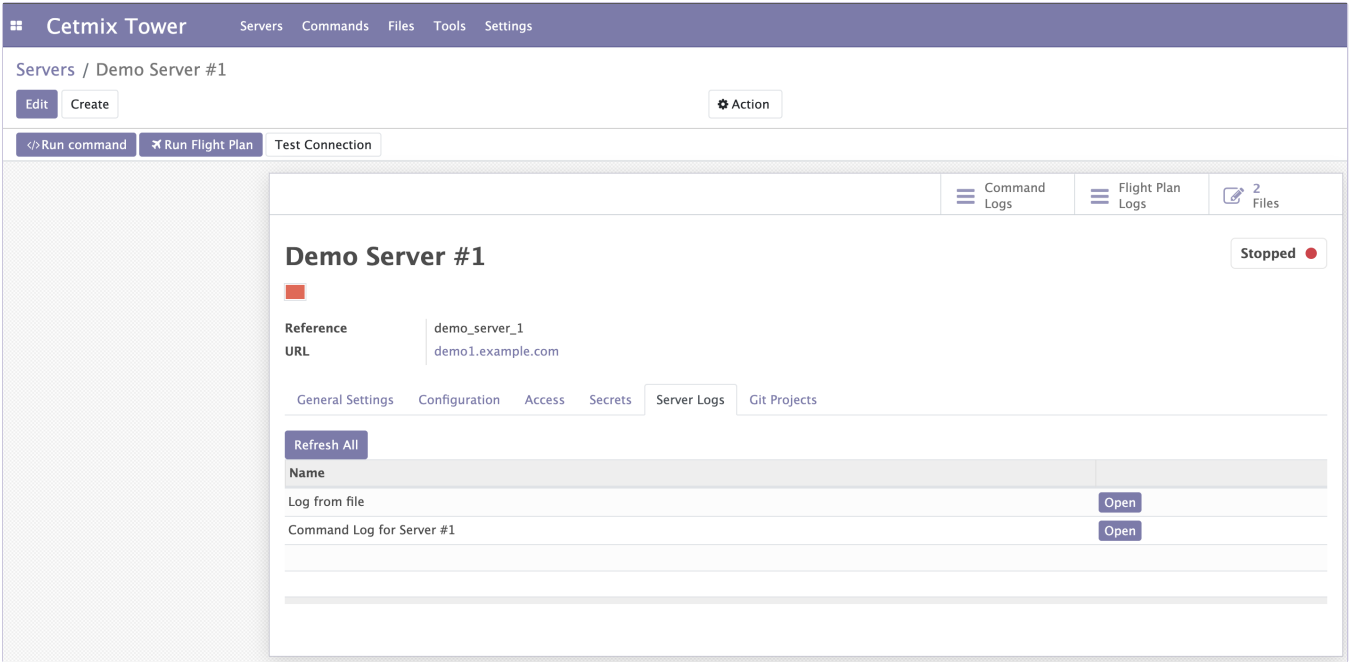


Figura 4.2 — Pestaña Server Logs con dos ventanas configuradas. El botón **Refresh All** las actualiza todas; **Open** abre una en pantalla completa.

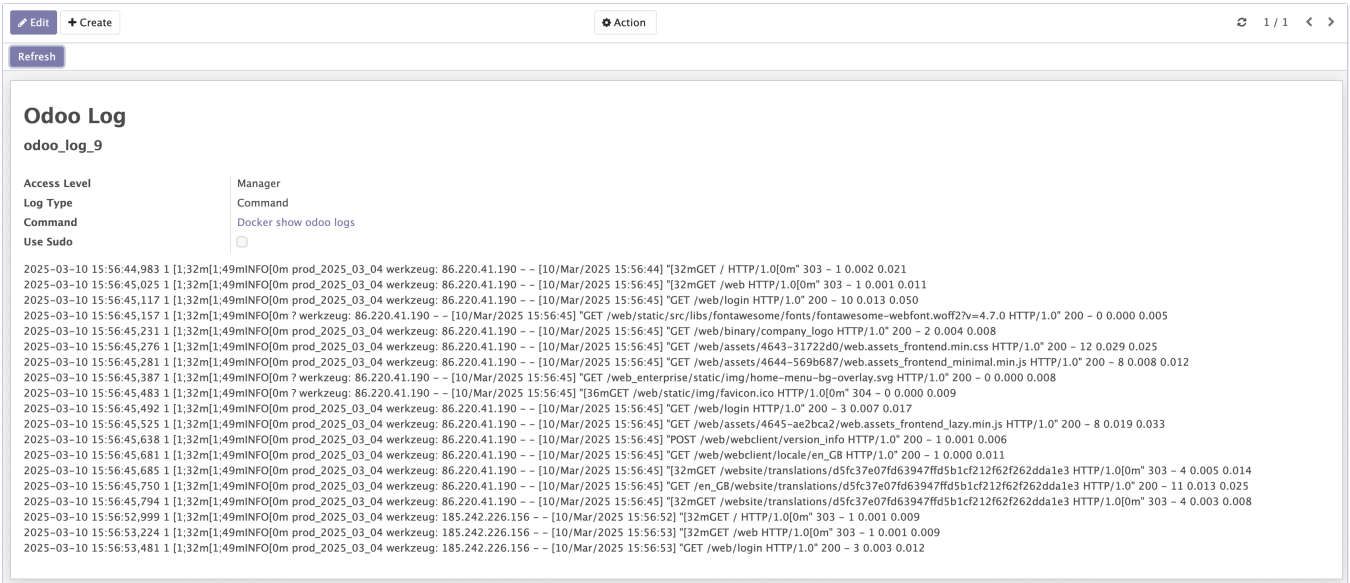


Figura 4.3 — Una ventana de log de tipo Command ya abierta, mostrando la salida de `docker logs` de una instancia Odoo. Gracias a `ansi2html`, los colores de la salida se conservan. El botón **Refresh** vuelve a ejecutar el comando.

TRUCO DE SOPORTE

Crea en cada jet una ventana de log de tipo Command con `docker logs --tail 200 odoo-{{ instance_name }}` y nivel de acceso `User`. Tu equipo de soporte N1 podrá diagnosticar la instancia de un cliente desde Odoo, sin darle acceso SSH al servidor.

Comandos: la unidad de trabajo

Un comando es una tarea con nombre, reutilizable y auditada. Todo lo que Tower hace acaba siendo un comando: los flight plans los encadenan, los jets los disparan y las tareas programadas los repiten. Merece la pena entender bien sus seis tipos.

5.1 Los seis tipos de acción

El campo **Action** de `Cetmix Tower > Commands > Commands` determina qué hace el comando:

Acción	¿Dónde corre?	Descripción
SSH Command <code>ssh_command</code>	En el servidor remoto	El caballo de batalla. Ejecuta el contenido del campo Code en una shell del servidor. El código de salida del proceso es el resultado del comando.
Python Code <code>python_code</code>	En el servidor de Tower	Ejecuta Python dentro de Odoo, no en la máquina remota. Sirve para hablar con APIs (OVH, AWS, Stripe), leer o escribir en la base de datos de Odoo y tomar decisiones.
Create/Update File <code>file_using_template</code>	Sube al servidor	Renderiza una plantilla de archivo con las variables del contexto y la deposita en el servidor. El campo If file exists decide si omitir, sobrescribir o fallar.
Run Flight Plan <code>plan</code>	Orquestación	Lanza otro flight plan. Permite componer procesos: un plan "desplegar" que llama a "instalar" y luego a "configurar DNS".
Trigger Jet Action <code>jet_action</code>	Orquestación	Provoca una transición de estado en un jet. Así un plan puede, por ejemplo, detener una instancia antes de hacerle backup y volver a arrancarla al terminar.
Create Waypoint <code>create_waypoint</code>	Orquestación	Crea un punto de restauración del jet (ver 9.7). El uso natural es el snapshot previo a una actualización.

LA CONFUSIÓN MÁS FRECUENTE

Un comando **Python Code** no se ejecuta en el servidor remoto. Corre dentro del proceso de Odoo que hospeda Tower. La propia interfaz lo advierte con un aviso amarillo. Si quieres ejecutar Python *en el servidor*, usa un comando SSH que invoque `python3` allí.

5.2 Campos comunes de un comando

Campo	Para qué sirve
Name / Reference	Nombre visible e identificador técnico estable
Access Level	Quién puede verlo y ejecutarlo. Los comandos destructivos deberían ser Manager o Root.
Default Path	Directorio de trabajo. Admite variables , por ejemplo <code>{{ base_dir }}/{{ instance_name }}</code> . (En el diálogo de ejecución manual, en cambio, el campo <i>Path</i> no las admite.)
Servers	Si lo rellenas, el comando solo aparece para esos servidores. Vacío = disponible en todos.
Operating Systems	Restringe a ciertos SO. Ejecutarlo en otro devuelve <code>-207</code> .
Tags	Clasificación y filtrado
Allow Parallel Run	Desmarcado (por defecto), un segundo lanzamiento en el mismo servidor se rechaza con <code>-201</code> mientras el primero corre. Es la protección natural contra el doble clic.
Use sudo / No split for sudo	Elevación de privilegios. La segunda casilla evita que Tower parta el comando en varias líneas al aplicar sudo, útil en scripts con <code>&&</code> o tuberías.
Variables	Pestaña de solo lectura: Tower detecta automáticamente las variables usadas en <code>code</code> y <code>path</code> y las lista aquí. Excelente para validar que no te falta ninguna.

5.3 Ejecutar un comando a mano

Desde la ficha de un servidor, pulsa **Run command**.

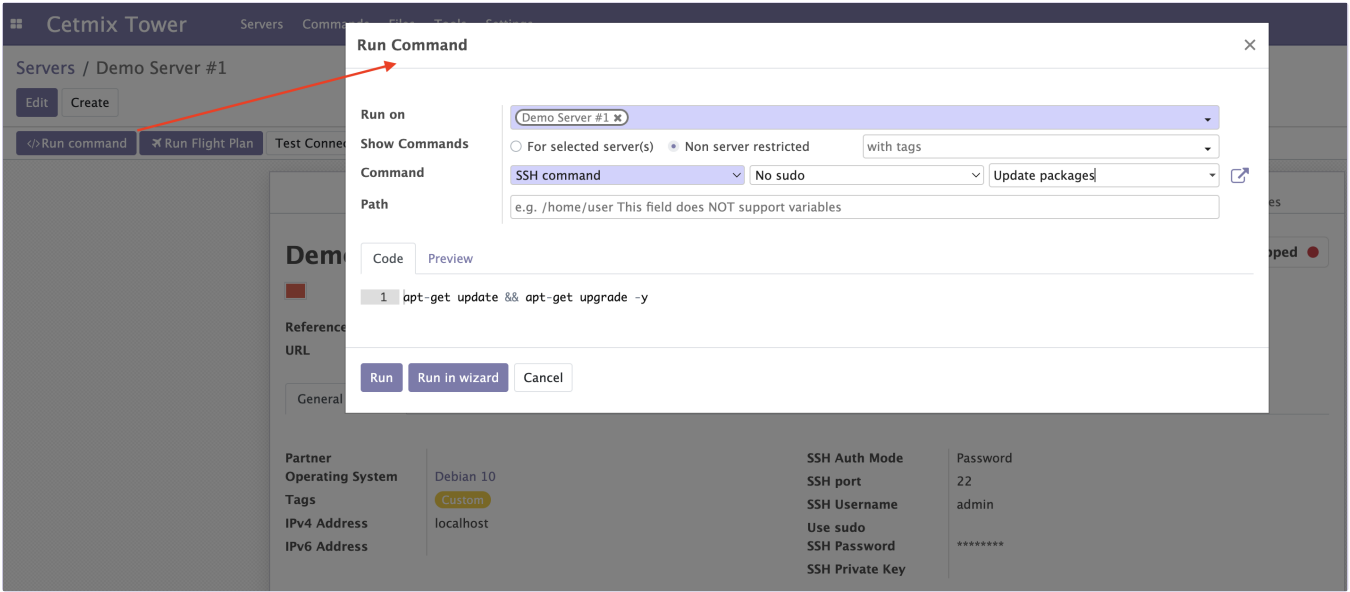


Figura 5.1 — Diálogo **Run Command**. **Run on** acepta varios servidores a la vez. El primer desplegable filtra por tipo de acción, el segundo por sudo y el tercero elige el comando. La pestaña **Preview** muestra el código ya renderizado con las variables sustituidas: **revísala siempre antes de ejecutar algo destructivo**.

Los dos botones no hacen lo mismo:

Botón	Comportamiento
Run	Lanza y cierra el diálogo. El resultado llega como notificación y queda en el log.
Run in wizard	Se queda en pantalla mostrando la salida en vivo. Es el modo adecuado para depurar un comando nuevo.

5.4 Comandos Python: el escape hacia el resto del mundo

Un comando Python devuelve su resultado asignando el diccionario `result`:

```
result = {"exit_code": 0, "message": "Todo bien"}
```

Clave	Significado
exit_code	Entero. 0 es éxito; cualquier otro valor es fallo y activa las acciones de error del flight plan. Por defecto, 0.
message	Texto que se guarda en el log del comando. Por defecto, None.

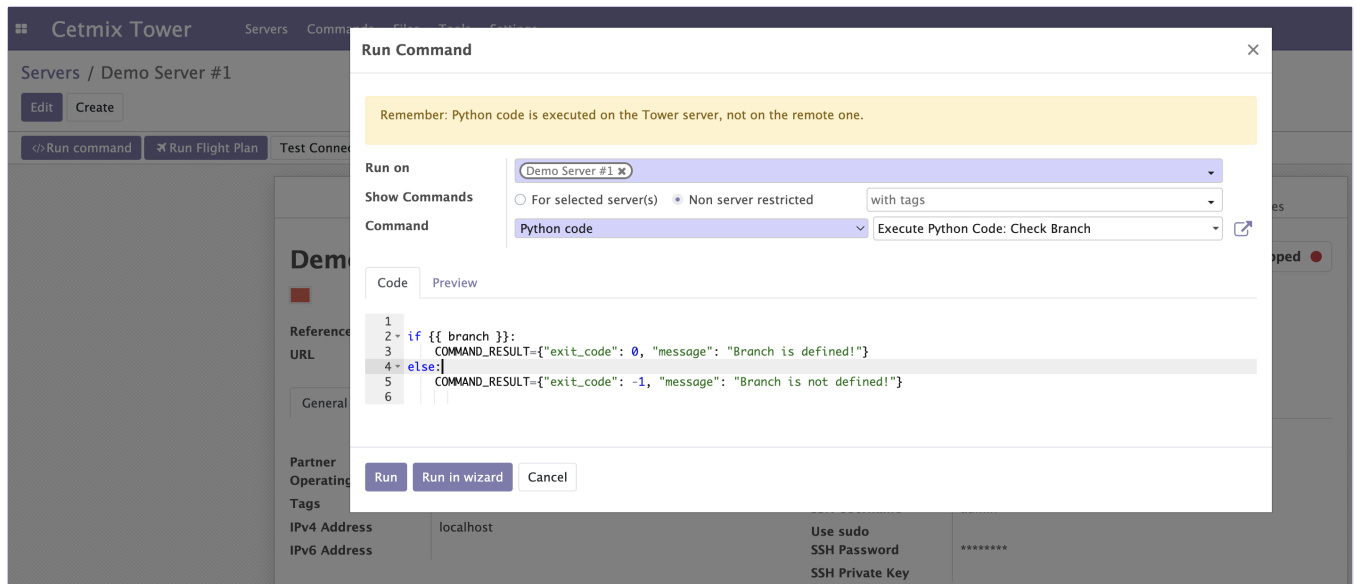


Figura 5.2 — Un comando Python en el diálogo de ejecución. Arriba, el aviso de que el código corre en el servidor de Tower. Nótese que las variables `{{ }}` se pueden usar dentro del código Python.

Advertencia: esta captura procede de documentación de una versión anterior y usa `COMMAND_RESULT`. En la versión 18.0 la variable correcta es `result`.

Qué tienes disponible dentro del código

Objeto	Uso
<code>server</code>	El registro <code>cx.tower.server</code> sobre el que se ejecuta
<code>custom_values</code>	Diccionario compartido entre los comandos de un mismo flight plan . Es el mecanismo para pasar datos de un paso al siguiente. No se persiste en base de datos.
<code>result</code>	El diccionario de salida
<code>ovh</code>	Cliente OVHcloud, si <code>cetmix_tower_ovh</code> está instalado
<code>boto3</code>	SDK de AWS, si <code>cetmix_tower_aws</code> está instalado
Objetos de Odoo	Un conjunto acotado de modelos y utilidades. La pestaña Help del comando enumera exactamente lo que hay disponible en tu instalación.

RESTRICCIONES DE SEGURIDAD

El intérprete filtra palabras clave peligrosas (importaciones arbitrarias, acceso al sistema de archivos del servidor de Odoo). Si un fragmento legítimo se bloquea, la salida lo indica: replantéalo con las librerías ya expuestas en lugar de intentar sortear el filtro.

Ejemplo: pasar datos entre pasos de un plan

```
# Paso 1 (Python): calcula el puerto libre y lo deja en custom_values
usados = server.jet_ids.mapped("variable_value_ids").filtered(
    lambda v: v.variable_id.reference == "odoo_http_port"
).mapped("value_char")
puerto = 10069
while str(puerto) in usados:
    puerto += 100
custom_values["odoo_http_port"] = str(puerto)
result = {"exit_code": 0, "message": "Puerto asignado: %s" % puerto}
```

5.5 Logs de comando

Cada ejecución deja un registro en **Cetmix Tower > Logs > Command Log** con el comando renderizado, el código de salida, la salida estándar, el error, la duración, el servidor, el jet y el usuario. Es tu pista de auditoría: quién ejecutó qué, dónde y con qué resultado.

Los comandos que superan el **Command Timeout** quedan "zombis" hasta que el cron `Check zombie commands` los cierra con el código `-206`.

Archivos y plantillas de archivo

Todo el aprovisionamiento de una instancia Odoo se reduce, en el fondo, a depositar tres archivos de configuración correctos en el servidor. Este es el subsistema que lo hace, y el que convierte una plantilla genérica en la configuración concreta de un cliente.

6.1 Plantilla contra archivo

Objeto	Qué es
File Template <code>cx.tower.file.template</code>	El molde. Contiene el nombre del archivo, el directorio destino y el cuerpo con variables <code>{{ }}</code> . Se define una vez y se reutiliza para todos los clientes.
File <code>cx.tower.file</code>	Una instancia concreta: esta plantilla, renderizada con estas variables, para este servidor o este jet, depositada en esta ruta. Guarda además una copia del contenido actual en el servidor.

6.2 Crear una plantilla

Ve a **Cetmix Tower > Files > Templates** y pulsa **Nuevo**.

Campo	Detalle
Name / Reference	Nombre y referencia estable de la plantilla
File name	Nombre del archivo sin ruta . Admite variables: <code>{{ instance_name }}</code> .conf
Directory on server	Ruta del directorio destino. También admite variables: <code>{{ base_dir }}</code> / <code>{{ instance_name }}</code>
File content	El cuerpo. Aquí van las variables y, si hace falta, los secretos.
File Type	<i>Text</i> o <i>Binary</i> . Los binarios se guardan como adjunto y no se renderizan.
Source	<i>Tower</i> : la verdad está en Odoo y se empuja al servidor. <i>Server</i> : la verdad está en el servidor y Tower la lee.
Keep when deleted	Si se marca, al borrar el registro en Tower el archivo permanece en el servidor. Márcalo para archivos de datos; déjalo sin marcar para configuración generada.
Auto Sync	Activa la descarga periódica del archivo desde el servidor (solo tiene sentido con <i>Source = Server</i>)
Variables	Detectadas automáticamente a partir del contenido, el nombre y el directorio

La dirección del flujo: Tower o Server

Source = Tower

Odoo es la fuente de verdad. Editas el contenido en Odoo y pulsas "Push to Server".

Para: `docker-compose.yml`,
`odoo.conf`, `vhosts` de `nginx`

Source = Server

El servidor es la fuente de verdad. Tower descarga el archivo y lo muestra en Odoo (Pull from Server).

Para: logs, informes generados,
archivos que produce la app

Cetmix Tower						
Files Create Files Templates Search... Filters Group By Favorites 1-9 / 9						
Rendered Name	Source	File Type	File	Server	Last Sync Date	Template
☐ staging_tower_demo_2.txt	Tower	Text		Demo Server #2		Demo File Template 2
☐ repos.yaml	Tower	Text		Demo Server #1		
☐ server_demo_logs.txt	Server	Text		Demo Server #2		Demo File Template 4
☐ server_demo_logs.txt	Server	Text		Demo Server #2		Demo File Template 4
☐ server_demo_logs.txt	Server	Text		Demo Server #1		Demo File Template 4
☐ server_demo.txt	Server	Text		Demo Server #2		
☐ tower_demo_1.txt	Tower	Text		Demo Server #2		Demo File Template 1
☐ tower_demo_3.txt	Tower	Text		Demo Server #2		Demo File Template 3
☐ tower_demo_without_template_staging.txt	Tower	Text		Demo Server #2		

Figura 6.1 — Listado de archivos. La columna **Source** distingue los que empuja Tower de los que lee del servidor; **Rendered Name** muestra el nombre ya con las variables resueltas, y **Last Sync Date** cuándo se sincronizó por última vez.

Cetmix Tower

Servers Commands Files Tools Settings

Servers / Demo Server #2 / Files / server_demo.txt

Edit Create

Action

Pull from Server

server_demo.txt

Source

File Type

Template

Server

Directory on server

Full Server Path

Server

Text

Demo Server #2

/opt/demo_server_2/cetmix-tower

/opt/demo_server_2/cetmix-tower/server_demo.txt

Auto Sync

Last Sync Date

Variables

☐

Demo Path

Code

Figura 6.2 — Ficha de un archivo con **Source** = **Server**. El botón **Pull from Server** trae la versión actual. **Full Server Path** es la ruta final calculada a partir del directorio y el nombre ya renderizados.

6.3 Acciones sobre un archivo

Acción	Efecto
Push to Server	Renderiza y sube por SFTP. Sobrescribe lo que hubiera.
Pull from Server	Descarga el contenido actual al campo <i>Code on server</i>
Get current server code	Lee sin guardar, para comparar antes de sobrescribir
Delete from Server	Borra el archivo en el servidor
Unlink from template	Desvincula el archivo de su plantilla para poder editarlo de forma independiente

Auto Sync

Con **Auto Sync** activo eliges un intervalo —10 min, 30 min, 1/2/6/12 horas, 1 día, 1 semana, 1 mes, 1 año— y el cron *Auto pull files from server* descarga el archivo cuando toca. Sirve para tener en Odoo una copia siempre fresca de un log o de un informe de estado sin abrir sesiones SSH manuales.

6.4 Las tres plantillas de la instancia Odoo efactusv

Estas son las plantillas concretas del kit de arranque. Créalas ahora: el capítulo 10 las usará.

Plantilla 1 — `docker-compose.yml`

Campo	Valor
File name	<code>docker-compose.yml</code>
Directory on server	<code>/opt/efactusv/{{ instance_name }}</code>
Source	Tower

```

services:
  odoo:
    container_name: odoo-{{ instance_name }}
    image: {{ odoo_image }}
    restart: always
    depends_on:
      - db
    networks:
      - net_{{ instance_name }}
    ports:
      - "127.0.0.1:{{ odoo_http_port }}:8069"
      - "127.0.0.1:{{ odoo_gevent_port }}:8072"
    volumes:
      - odoo-web-data:/var/lib/odoo
      - ./config:/etc/odoo
    environment:
      - HOST=db
      - USER=odoo
      - PASSWORD=#!cxtower.secret.db_password!#
      - PORT=5432
    entrypoint: "/usr/bin/odoo -c /etc/odoo/odoo.conf"

  db:
    image: postgres:16
    container_name: db-{{ instance_name }}
    restart: always
    environment:
      POSTGRES_USER: odoo
      POSTGRES_PASSWORD: #!cxtower.secret.db_password!#
      POSTGRES_DB: postgres
    networks:
      - net_{{ instance_name }}
    volumes:
      - odoo-db-data:/var/lib/postgresql/data

volumes:
  odoo-db-data:
  odoo-web-data:

networks:
  net_{{ instance_name }}:
    driver: bridge

```

PUERTOS LIGADOS A LOCALHOST

Fíjate en "127.0.0.1:{{ odoo_http_port }}:8069". Sin el prefijo 127.0.0.1, Docker publicaría el puerto en todas las interfaces y cualquiera podría saltarse nginx —y el SSL— entrando directamente por `http://ip-del-servidor:10069`. El prefijo no es opcional.

Plantilla 2 — `odoo.conf`

Campo	Valor
File name	<code>odoo.conf</code>
Directory on server	<code>/opt/efactusv/{{ instance_name }}/config</code>

```
[options]
addons_path =
    /mnt/extra-addons,
    /mnt/efactusv,
    /mnt/contract,
    /mnt/knowledge,
    /mnt/dms
admin_passwd = #!cxtower.secret.admin_passwd!#
db_host = db
db_port = 5432
db_user = odoo
db_password = #!cxtower.secret.db_password!#
db_name = {{ instance_name }}
dbfilter = ^{{ instance_name }}$
list_db = False
proxy_mode = True
workers = {{ odoo_workers }}
max_cron_threads = 1
gevent-port = 8072
limit_time_cpu = 600
limit_time_real = 1200
limit_request = 8192
transient-age-limit = 1.0
log_level = info
```

LAS TRES LÍNEAS QUE AÍSLAN AL CLIENTE

`dbfilter = ^{{ instance_name }}$` hace que la instancia solo sirva su propia base de datos. `list_db = False` oculta el selector de bases. `proxy_mode = True` hace que Odoo confíe en las cabeceras `X-Forwarded-*` de nginx, imprescindible para que las URLs generadas y las cookies seguras funcionen tras el proxy. Las tres juntas, o el aislamiento es aparente.

Plantilla 3 — vhost de nginx

Campo	Valor
File name	<code>{{ instance_name }}.conf</code>
Directory on server	<code>/etc/nginx/sites-enabled</code>

```

upstream odoo_{{ instance_name }} {
    server 127.0.0.1:{{ odoo_http_port }};
}
upstream gevent_{{ instance_name }} {
    server 127.0.0.1:{{ odoo_gevent_port }};
}

server {
    listen 80;
    server_name {{ instance_domain }};

    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }
    location / {
        return 301 https://{{ instance_domain }}$request_uri;
    }
}

server {
    listen 443 ssl;
    server_name {{ instance_domain }};

    ssl_certificate      /etc/letsencrypt/live/{{ instance_domain }}/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/{{ instance_domain }}/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;

    client_max_body_size 512m;
    proxy_read_timeout 720s;
    proxy_connect_timeout 720s;
    proxy_send_timeout 720s;

    proxy_set_header X-Forwarded-Host {{ instance_domain }};
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Real-IP $remote_addr;

    location / {
        proxy_redirect off;
        proxy_pass http://odoo_{{ instance_name }};
    }

    location ~* /web/static/ {
        proxy_cache_valid 200 90m;
        proxy_buffering on;
        expires 864000;
        proxy_pass http://odoo_{{ instance_name }};
    }

    location /websocket {
        proxy_pass http://gevent_{{ instance_name }};
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header X-Forwarded-Host {{ instance_domain }};
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Real-IP $remote_addr;
    }

    gzip_types text/css text/less text/plain text/xml application/xml
        application/json application/javascript;

    gzip on;
}

```

ORDEN DE OPERACIONES CON SSL

Este vhost referencia certificados que todavía no existen la primera vez, así que `nginx -t` fallará si lo despliegas antes de emitirlos. El flight plan del capítulo 10 lo resuelve emitiendo el certificado con el bloque HTTP-01 ya activo **antes** de recargar nginx con el bloque 443.

Flight Plans: procesos reproducibles

Un flight plan es una lista ordenada de comandos con lógica de decisión entre ellos. Es lo que convierte "diez comandos que hay que ejecutar en orden y rezar" en un proceso que se puede repetir mil veces con el mismo resultado y con un registro de lo ocurrido.

7.1 Anatomía de un plan

Ve a [Cetmix Tower > Commands > Flight Plans](#).

Campo	Detalle
Name / Reference	Identificación
Allow Parallel Run	Si está desmarcado, dos ejecuciones simultáneas en el mismo servidor se rechazan con <code>-301</code>
On Error	Qué hacer si una línea falla y no tiene su propia acción definida: salir con el código del comando, salir con un código personalizado, o continuar
Servers / Tags	Restricción de alcance y clasificación
Access Level	Tower avisa si el plan contiene comandos de nivel superior al del propio plan
Code (pestaña)	Las líneas del plan: la secuencia real

Flight Plan / Demo Flight Plan #1

[Edit](#) [Create](#) [Action](#) 1 / 5

Name
Demo Flight Plan #1

Reference
demo_flight_plan_1

Allow Parallel Run
☐

On Error
Exit with command exit code

Note
Create directory and list its content

Servers
Tags
Access Level
Manager

[Code](#) [Access](#) [YAML](#)

Name	Action	Tags	Use Sudo	Path	Condition
Create directory	SSH command	Production	☐	/such/much/{{ demo_path }}	
List files in directory	SSH command	Custom	☐		{{ tower.server.st
Upload file by template	Create file using template	Staging	☐		

Figura 7.1 — Ficha de un flight plan. En la pestaña `Code`, cada línea muestra su comando, la acción, si usa `sudo`, la ruta (con variables) y la condición de ejecución.

7.2 Las líneas del plan

Campo de la línea	Detalle
Sequence	Orden de ejecución. Deja huecos (5, 10, 15, 20...) para poder insertar pasos después sin reenumerar todo.
Command	El comando a ejecutar
Path	Sobrescribe el directorio de trabajo para esta línea. Admite variables.
Use sudo	Elevación solo para esta línea
Condition	Expresión que decide si la línea se ejecuta o se salta. Si no se cumple, la línea se registra como omitida con el código <code>-205</code> y el plan continúa.
Post Run Actions	Las reglas de decisión según el código de salida: la parte interesante

Post Run Actions: el árbol de decisión

Cada línea puede tener varias reglas, evaluadas en orden. Una regla es «**si el código de salida <condición> <valor> entonces <acción>**».

Elemento	Opciones
Condition	<code>==</code> <code>!=</code> <code>></code> <code>>=</code> <code><</code> <code><=</code>
Result	El valor con el que se compara el código de salida
Action	<i>Run next command</i> : continúa con la línea siguiente. <i>Exit with command exit code</i> : aborta el plan devolviendo el código del comando. <i>Exit with custom exit code</i> : aborta devolviendo el código que tú indiques.
Variable Values	Valores de variables que se fijan al aplicarse esta regla, disponibles para las líneas siguientes

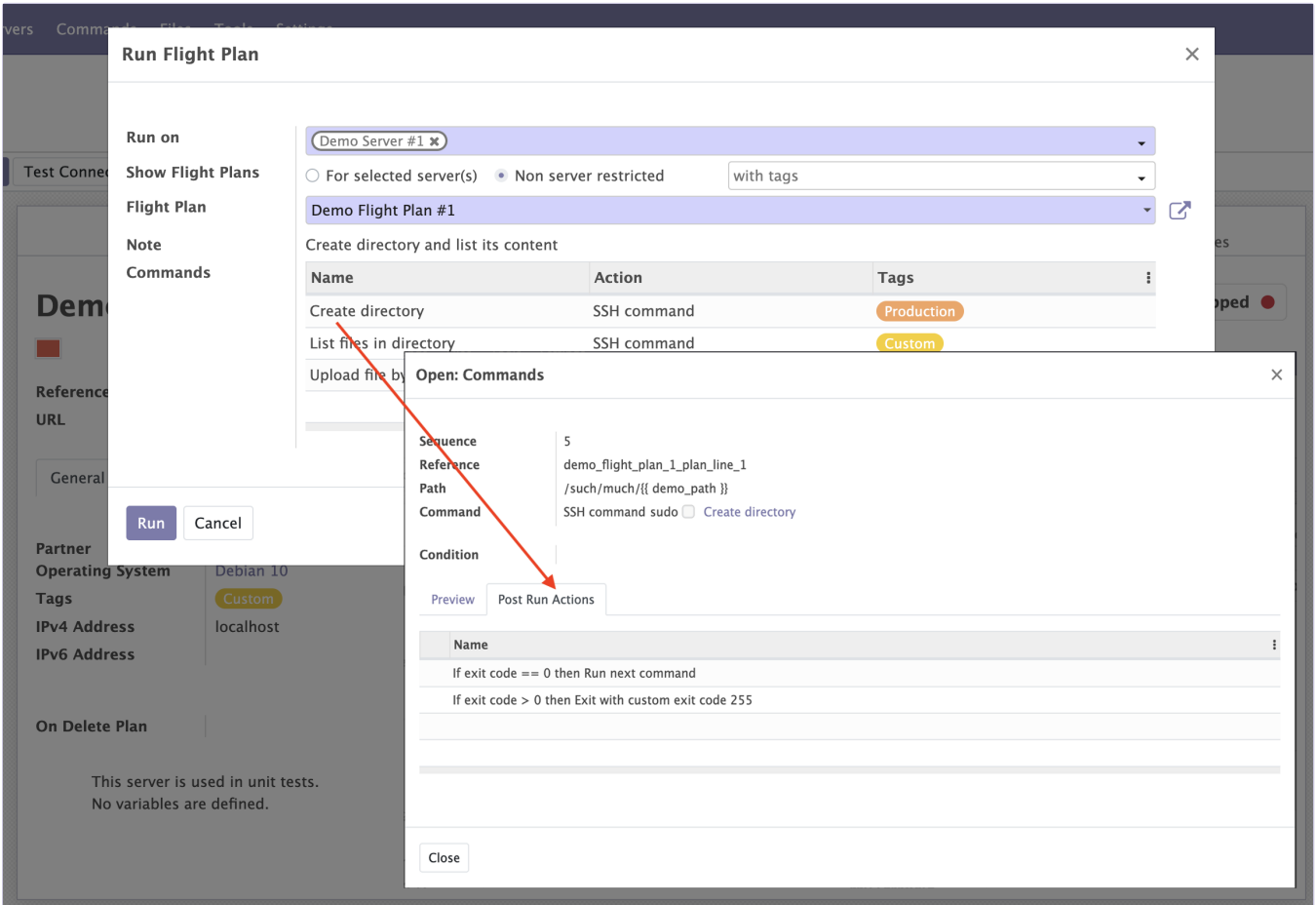


Figura 7.2 — La pestaña *Post Run Actions* de una línea. Las dos reglas visibles forman el patrón más común: «si exit code == 0 entonces ejecutar el siguiente comando» y «si exit code > 0 entonces salir con el código personalizado 255»

EL PATRÓN "FAIL FAST" QUE DEBERÍAS USAR POR DEFECTO

En un plan de aprovisionamiento, cualquier fallo debe abortar: seguir adelante deja al cliente con una instancia a medio construir, que es peor que ninguna. Configura **On Error = Exit with command exit code** a nivel de plan y no añadas reglas por línea salvo en los pasos que sí toleran fallo (por ejemplo, un `mkdir` de un directorio que quizá ya existe).

7.3 Ejecutar un plan

Desde la ficha del servidor, **Run Flight Plan**.

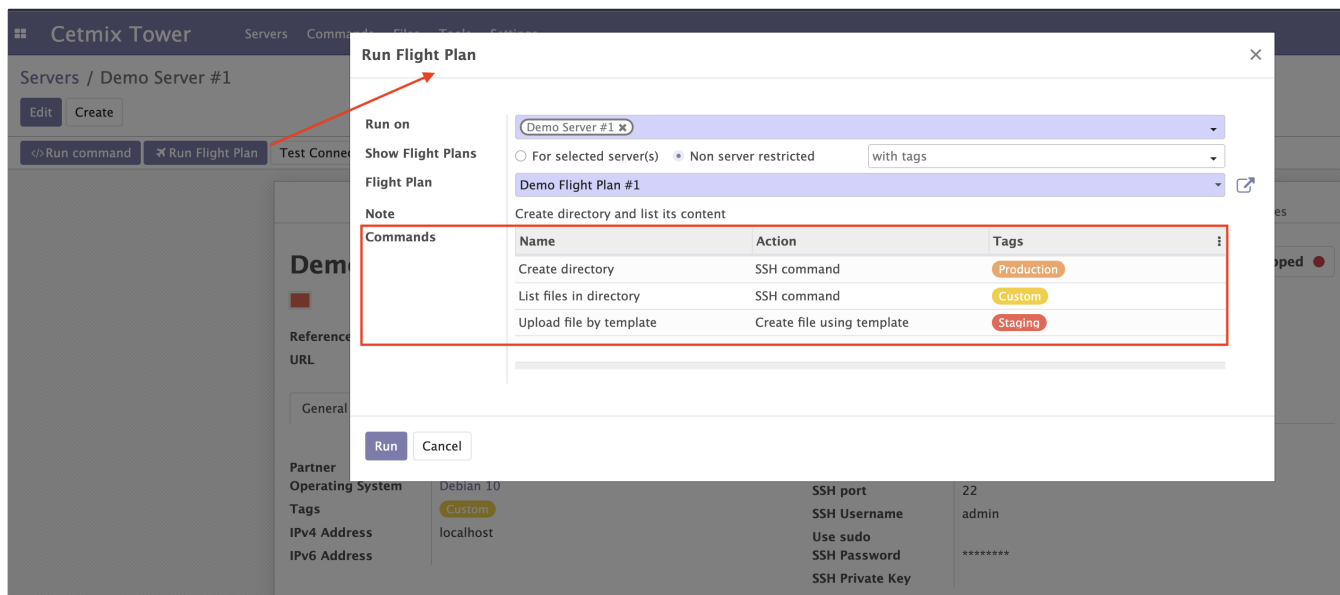


Figura 7.3 — Diálogo Run Flight Plan. Antes de lanzar, el recuadro inferior enumera los comandos que se van a ejecutar en orden, con su tipo de acción y sus etiquetas. Revísalo: es tu última oportunidad de comprobar que el plan es el que crees.

El seguimiento está en **Cetmix Tower > Logs > Flight Plan Log**. Cada ejecución guarda el estado global, la línea en curso, y un log de comando por cada paso, con su salida completa.

7.4 Anidar planes

Como un comando puede tener acción **Run Flight Plan**, los planes se componen. Tower detecta y rechaza la recursión, así que no puedes crear un bucle infinito por accidente. La estructura que funciona bien para efactusv:

```
efactusv instance install      ← plan maestro, invocado por el jet
├─ (SSH)  crear estructura de directorios
├─ (File)  subir docker-compose.yml
├─ (File)  subir odoo.conf
├─ (SSH)  docker compose pull && up -d
├─ (Plan)  efactusv_instance_dns ← subplan reutilizable
│   └─ (Python) crear registro A en OVH
│   └─ (SSH)   esperar propagación DNS
├─ (Plan)  efactusv_instance_ssl ← subplan reutilizable
│   └─ (File)  subir vhost de nginx
│   └─ (SSH)  certbot certonly
│   └─ (SSH)  nginx -t && systemctl reload nginx
└─ (SSH)  inicializar la base de datos
```

Los subplanes `_dns` y `_ssl` se reutilizan luego, tal cual, en el proceso de "añadir dominio propio del cliente" del capítulo 11.

7.5 Códigos de salida de Tower

Además de los códigos del sistema operativo (0 = éxito), Tower usa códigos negativos propios. Conocerlos convierte un fallo opaco en un diagnóstico inmediato.

Código	Significado	Qué revisar
-100	Error general	El log de Odoo
-101	Recurso no encontrado	Referencias mal escritas
503	Error de conexión SSH	Credenciales, host key, red
-201	Ya hay otra instancia del comando corriendo	Espera, o activa <i>Allow Parallel Run</i>
-202	No hay ejecutor para esa acción	Instalación incompleta
-203	Error en el código Python	La traza en el log del comando
-205	La condición de la línea no se cumplió (línea omitida)	Normal, no es un fallo

Código	Significado	Qué revisar
-206	El comando expiró y fue terminado	Sube <i>Command Timeout</i>
-207	Comando incompatible con el SO del servidor	El campo <i>Operating Systems</i> del comando
-208	Comando detenido por el usuario	—
-301	Ya hay otra ejecución del plan corriendo	Espera o permite paralelismo
-302	El plan no tiene líneas	Plan vacío
-306	Algún comando del plan es incompatible con el servidor	Revisa los SO de cada comando
-308	Plan detenido por el usuario	—
-400	No se pudo crear el archivo en el servidor	Permisos del directorio destino
-401 / -402	Fallo al subir / descargar archivo	SFTP, espacio en disco, permisos
-501 ... -508	Errores del subsistema de jets	Ver la tabla del capítulo 9

Plantillas de servidor: alta de VPS repetible

Cuando crezcas de un servidor a diez, no querrás rellenar quince campos a mano cada vez. Una plantilla de servidor captura toda esa configuración —conexión, variables, logs, tareas programadas— y añade algo más potente: un flight plan que se ejecuta solo al crear el servidor.

8.1 Qué copia una plantilla

En `Cetmix Tower > Servers > Templates`, una plantilla arrastra al nuevo servidor:

Bloque	Contenido
Conexión	Puerto SSH, usuario, modo de autenticación, contraseña, llave privada, uso de sudo
Atributos	Color, sistema operativo, etiquetas
Variables	Todos los valores de la pestaña <i>Configuration</i> , incluidos los marcados como requeridos
Server Logs	Las ventanas de log preconfiguradas. Si alguna es de tipo archivo, Tower crea el archivo correspondiente en el servidor nuevo.
Shortcuts	Accesos directos a comandos frecuentes
Scheduled Tasks	Tareas programadas asociadas
Flight Plan	Plan que se ejecuta automáticamente justo después de crear el servidor
On Delete Plan	Plan que se ejecuta al eliminar el servidor. Campo reservado a Managers y que no puede sobrescribirse desde la API: siempre viene de la plantilla.

Server Templates / Demo Server Template #1

Edit Create

⚙️ Action

1 / 1 < >

Create Server

1 Servers



Demo Server Template #1

Custom

Reference demo_server_template_1

General Settings Configuration Access Server Logs YAML

Flight Plan
On Delete Plan
Operating System

Demo Flight Plan #1
Demo Flight Plan for User
Debian 10

SSH Auth Mode
SSH port
SSH Username
Use sudo
SSH Password
SSH Private Key

Password
22
admin
Without password

Key 1 SSH

Send message Log note ⌚ Schedule activity

🔒 0 Follow 👤 1

Figura 8.1 — Ficha de una plantilla de servidor. A la izquierda, los dos flight plans del ciclo de vida y el SO; a la derecha, los parámetros de conexión que heredarán todos los servidores creados con ella.

8.2 Crear un servidor desde la plantilla

Guía Cetmix Tower · efactusv

31

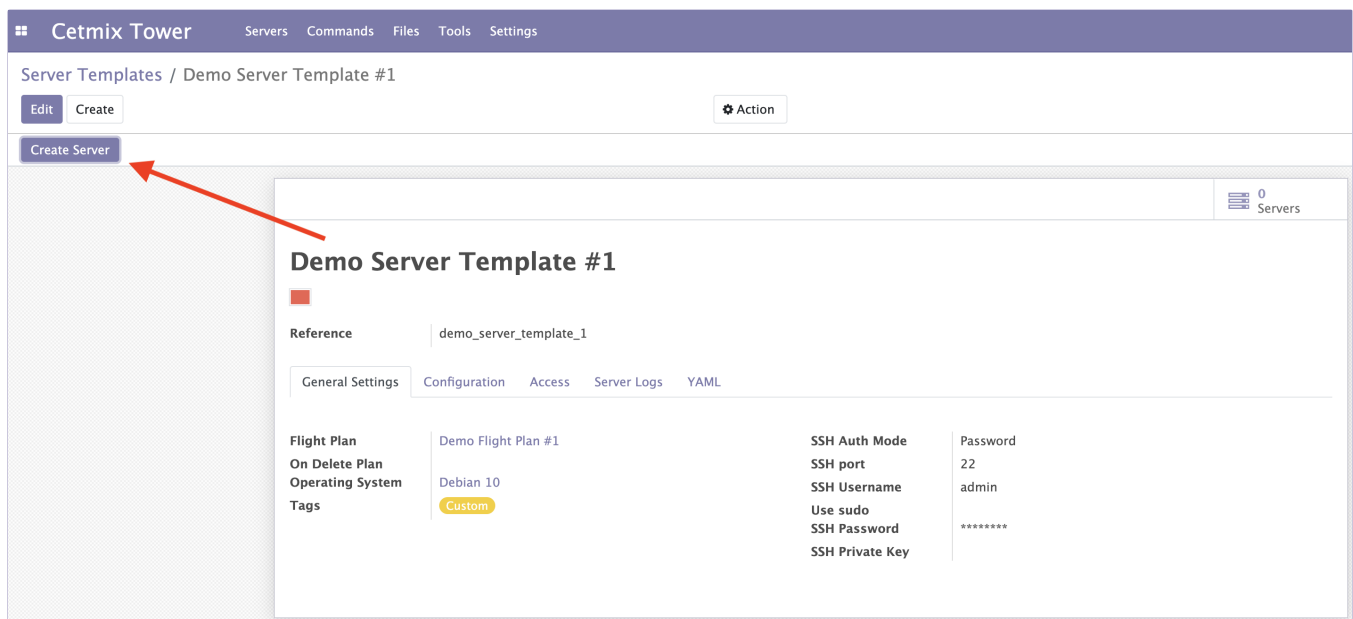


Figura 8.2 — El botón **Create Server** de la plantilla abre un asistente donde solo tienes que dar el nombre, la IP y los valores de las variables que la plantilla marcó como requeridas. El contador **Servers** lleva la cuenta de las máquinas nacidas de esta plantilla.

El asistente valida las **variables requeridas** antes de crear nada: si falta alguna o viene vacía, muestra un error con la lista exacta de referencias que faltan. Así no acabas con servidores a medio configurar.

8.3 Crear servidores desde código

Esta es la puerta de entrada a la automatización total —un webhook, una acción sobre un pedido de venta, un comando Python—. El método es `create_server_from_template`:

```
server = env["cx.tower.server.template"].create_server_from_template(
    "efactusv_prod_vps",          # referencia de la plantilla
    "PROD-SV-04",                # nombre del nuevo servidor
    partner=partner,              # res.partner opcional
    ipv4="203.0.113.42",
    ssh_key="ssh_prod",           # referencia de la llave
    configuration_variables={
        "base_dir": "/opt/efactusv",
        "odoo_workers": "4",
    },
)
```

Parámetro	Detalle
partner	Registro <code>res.partner</code> propietario
ipv4 / ipv6	Al menos una de las dos
ssh_password / ssh_key	Credencial; <code>ssh_key</code> es la <i>referencia</i> del registro de llave, no su contenido
configuration_variables	Diccionario {referencia: valor}. Si una referencia no existe como variable, Tower la crea.
configuration_variable_options	Para variables de tipo <i>Options</i> : {referencia_variable: referencia_opcion}. Si la opción no existe, se lanza un error detallado.
pick_all_template_variables	True (por defecto) fusiona las variables de la plantilla con las que pasas. False usa solo las que pasas.

UN MATIZ IMPORTANTE DE LA VALIDACIÓN

Cuando `pick_all_template_variables` es True, la comprobación de variables requeridas se hace únicamente si envías `configuration_variables`. Si no envías ninguna, se asume que las de la plantilla bastan. Tenlo presente al automatizar: pasa siempre explícitamente las variables que definen la identidad del servidor.

Action Name

Create new server

Model: Sales Order

Action To Do: Execute Python Code

Active: ☒

Trigger: On Update

Trigger Fields:

Before Update Domain: Match records with the following rule:

Status in "draft" or "sent"

→ 4 record(s) [Edit Domain](#)

Apply on: Match all records

→ 21 record(s) [Edit Domain](#)

Figura 8.3 — El mismo método invocado desde una acción de servidor de Odoo (Ajustes > Técnico > Acciones de servidor), disparada al confirmarse un pedido de venta. Este es el patrón "el cliente compra y el servidor se aprovisiona solo".

Jets: el motor SaaS

Aquí está la diferencia entre "Odoos que ejecuta comandos SSH" y "una plataforma SaaS". Un jet es una aplicación con ciclo de vida propio, que vive en un servidor, tiene estados, se mueve entre ellos mediante acciones, puede depender de otros jets y puede clonarse. Para efactusv, un jet es la instancia Odoos de un cliente.

9.1 El vocabulario de los jets

Objeto	Modelo	Papel
Jet State	<code>cx.tower.jet.state</code>	Un estado posible: <code>running</code> , <code>stopped</code> , <code>starting</code> ... Es global, se comparte entre plantillas.
Jet Action	<code>cx.tower.jet.action</code>	Una transición: de qué estado, a cuál, pasando por cuál mientras trabaja, y qué flight plan la realiza.
Jet Template	<code>cx.tower.jet.template</code>	El plano completo: estados, acciones, archivos, variables, planes de instalación y desinstalación, dependencias, límites.
Jet	<code>cx.tower.jet</code>	Una instancia concreta viviendo en un servidor concreto
Waypoint	<code>cx.tower.jet.waypoint</code>	Un punto de restauración: una versión guardada del jet a la que se puede "volar"
Jet Request	<code>cx.tower.jet.request</code>	Petición interna generada cuando un jet necesita que otro esté en cierto estado antes de continuar
Template Install	<code>cx.tower.jet.template.install</code>	Registro de la instalación (o desinstalación) de una plantilla en un servidor

9.2 Estados

Ve a `Cetmix Tower > Settings > Jets > States`. Un estado tiene nombre, referencia, secuencia (orden en las vistas), color y nivel de acceso. El color se propaga a las tarjetas kanban del jet, así que elige con criterio: verde para operativo, rojo para error.

Para el modelo efactusv necesitas seis:

Referencia	Tipo	Color	Significado
<code>running</code>	Estable	Verde	La instancia está en marcha y el cliente puede usarla
<code>stopped</code>	Estable	Gris	Contenedores parados; los datos siguen ahí
<code>starting</code>	Transitorio	Azul	Arrancando: el plan de start está corriendo
<code>stopping</code>	Transitorio	Azul	Deteniéndose
<code>installing</code>	Transitorio	Amarillo	Aprovisionamiento inicial en curso
<code>error</code>	Error	Rojo	Una transición falló; requiere intervención

ESTABLES, TRANSITORIOS Y DE ERROR

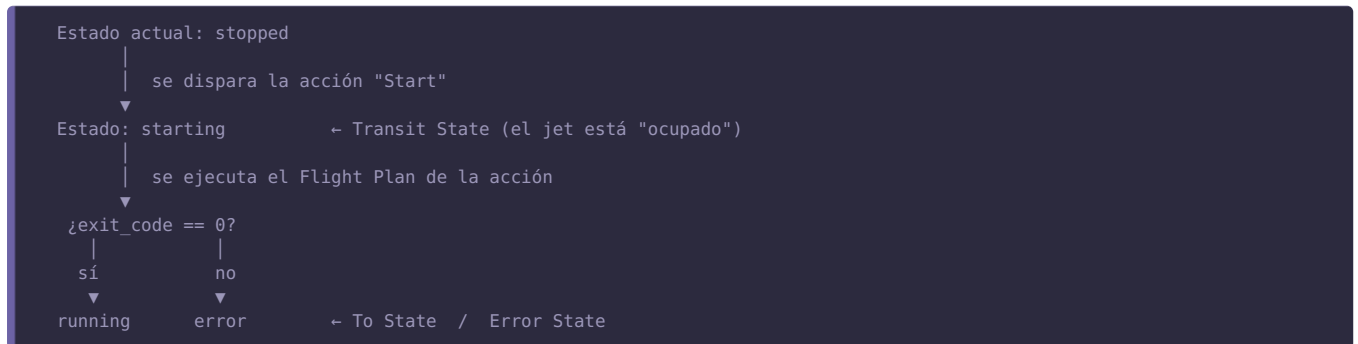
La distinción no es un campo del modelo, es una convención de diseño y es esencial. Los **estables** son donde el jet descansa. Los **transitorios** son donde está mientras un flight plan trabaja: sirven para que la interfaz muestre "arrancando..." y para impedir que se lancen dos operaciones a la vez. El de **error** es donde aterriza si algo sale mal.

9.3 Acciones: la máquina de estados

Una acción se define dentro de un jet template y tiene cinco campos clave:

Campo	¿Obligatorio?	Significado
From State	No	Estado de origen. Vacío significa "desde la nada" : es la acción de creación.
Transit State	Sí	Estado que ocupa el jet <i>mientras</i> el flight plan corre
To State	No	Estado de destino. Vacío significa "a la nada" : es la acción de destrucción.
Error State	Recomendado	Adónde va el jet si el plan devuelve un código distinto de cero
Flight Plan	Sí en la práctica	El plan que hace el trabajo real
Priority	—	Orden de evaluación cuando varias acciones podrían aplicarse

El ciclo de una transición



POR QUÉ EL TRANSIT STATE ES OBLIGATORIO

Mientras el jet está en un estado transitorio, Tower lo considera *ocupado* y rechaza nuevas acciones. Es lo que impide que un cliente impaciente pulse "Reiniciar" cinco veces y lance cinco `docker compose` simultáneos sobre la misma instancia.

Las acciones frontera: crear y destruir

Tower identifica automáticamente dos acciones especiales dentro de cada plantilla:

Acción	Cómo se identifica	Qué es en efactusv
Create action	La que tiene <i>From State</i> vacío	Aprovisionar la instancia: directorios, archivos, contenedores, DNS, SSL, base de datos
Destroy action	La que tiene <i>To State</i> vacío	Eliminar la instancia: bajar contenedores, borrar volúmenes, quitar el vhost, limpiar directorios

Cómo encuentra Tower el camino

Cuando pides llevar un jet a un estado que no es alcanzable en un solo paso, Tower construye el grafo de acciones y busca el camino más corto (recorrido en anchura). Si no existe camino, falla con un error explícito: «*No path found to bring the jet X to the state Y*». Después ejecuta las acciones en cadena, una tras otra.

```

Si defines: installing-running (Create),  running-stopped (Stop),
           stopped-running (Start),       stopped->∅ (Destroy)

Entonces "llevar de running a ∅" se resuelve solo como:
running → [Stop] → stopped → [Destroy] → ∅
  
```

9.4 El Jet Template

Ve a `Cetmix Tower > Jets > Templates`. Es el objeto más rico del sistema.

Campo / pestaña	Función
Name / Reference / Icon	Identidad. El icono aparece en las tarjetas de los jets.
Installation Flight Plan	Plan que prepara el servidor para alojar jets de este tipo (instalar Docker, crear directorios base, configurar nginx). Se ejecuta una vez por servidor, no una vez por jet.

Campo / pestaña	Función
Uninstallation Flight Plan	El inverso: limpia el servidor cuando retiras la plantilla
Clone Flight Plan (same server) Clone Flight Plan (different server)	Cómo duplicar un jet dentro del mismo servidor o hacia otro. La copia entre servidores requiere transferir datos, por eso son dos planes distintos.
Limit per server	Número máximo de jets de esta plantilla por servidor. Tu control de densidad: evita que un VPS acabe con cuarenta instancias.
Show in create wizard	Si aparece en el asistente <i>Launch New Jet</i>
Servers	Servidores donde la plantilla ya está instalada
States / Actions	La máquina de estados
Files	Plantillas de archivo que se materializan en cada jet
Variables	Valores por defecto para todos los jets nacidos de esta plantilla
Dependencies	Otras plantillas que deben estar presentes, y en qué estado
Waypoint Templates	Tipos de punto de restauración disponibles
Scheduled Tasks	Tareas periódicas que heredan los jets (backups)

El grafo de dependencias

La plantilla dibuja automáticamente un diagrama SVG con sus dependencias, calculando niveles y detectando ciclos. Una dependencia declara: «la plantilla A requiere que la plantilla B esté en el estado S» Tower **impide crear dependencias circulares** y no permite modificar una dependencia existente: hay que borrarla y crearla de nuevo.

Cuando un jet necesita que otro esté en cierto estado, Tower genera un **Jet Request** ([Logs > Jet Requests](#)) que atraviesa los estados *New* → *Processing* → *Success* / *Failed*. Es el mecanismo por el que, por ejemplo, un jet "Odo" puede exigir que el jet "PostgreSQL compartido" esté *running* antes de arrancar.

9.5 Instalar la plantilla en un servidor

Antes de crear jets, la plantilla debe estar **instalada** en el servidor: es cuando corre el *Installation Flight Plan* que deja la máquina lista.

1 Desde la plantilla

Botón **Install on servers** y selecciona los servidores. Tower omite los que ya la tienen o están instalándola, y avisa con una notificación.

2 Desde el servidor

Botón **Install Jet Template** en la ficha del servidor.

3 Seguimiento

[Logs > Jet Template Installations](#) muestra cada instalación con estado *Processing* / *Done* / *Failed*, la línea en curso y un enlace a los logs del flight plan.

REGLAS DE LA DESINSTALACIÓN

Tower se niega a desinstalar una plantilla de un servidor si (a) no está instalada, (b) todavía quedan jets de esa plantilla en la máquina, o (c) hay otras plantillas instaladas que dependen de ella. El orden correcto es siempre: destruir los jets primero, desinstalar la plantilla después.

9.6 Crear y operar jets

Desde la interfaz

[Cetmix Tower > Jets > Launch New Jet](#) abre el asistente: eliges plantilla, servidor, nombre y valores de variables. También hay un botón **New Jet** en la ficha del servidor.

Desde código

```
jet = jet_template.create_jet(
    server,                                # cx.tower.server
    name="cliente1",
    partner_id=partner.id,
    variable_values={
        "instance_name": "cliente1",
        "instance_domain": "cliente1.efactusv.com",
        "odoo_http_port": "10069",
        "odoo_gevent_port": "10072",
        "odoo_workers": "2",
    },
)
```

Si no das nombre, Tower genera uno legible con un sufijo aleatorio. Si el servidor ya alcanzó el **Limit per server**, el método devuelve `False` en lugar de lanzar una excepción: **comprueba siempre el resultado**.

CAMPOS ADMITIDOS AL CREAR UN JET

Por seguridad, `create_jet` solo acepta un conjunto blanco de campos: `name`, `reference`, `sequence`, `tag_ids`, `partner_id`, `jet_cloned_from_id`, `scheduled_task_ids` y `server_log_ids`. El módulo `efactusv_tower_contract` extiende esa lista para añadir `contract_id`; es exactamente el punto de extensión previsto si necesitas pasar más campos propios.

Operaciones sobre un jet

Operación	Detalle
Disparar una acción	Botón Action en la ficha. Solo se ofrecen las acciones válidas desde el estado actual.
Llevar a un estado	Botón State : eliges el destino y Tower calcula y ejecuta la cadena de acciones necesaria.
Ejecutar un comando	Con el contexto del jet cargado, de modo que <code>{{ instance_name }}</code> y <code>{{ tower.jet.* }}</code> resuelven a los valores de ese cliente
Ejecutar un flight plan	Igual, con contexto de jet
Clonar	Usa el plan de clonación correspondiente según el destino sea el mismo servidor u otro. El clon guarda referencia al original en <code>jet_cloned_from_id</code> .
Archivos y logs	Botones inteligentes a los archivos, logs de comando y logs de plan del jet

Desde Python: la API pública

```
# Preferible a _bring_to_state(): respeta los permisos del usuario
jet.bring_to_state("stopped")

# Ejecutar un comando o un plan en el contexto del jet
jet.run_command(command)
jet.run_flight_plan(plan)

# Leer una variable resuelta en cascada (jet → servidor → global)
dominio = jet.get_variable_value("instance_domain")

# Jets dependientes de otra plantilla
dependientes = jet.get_dependent_jets_by_template_reference("efactusv_postgres")
```

BRING_TO_STATE CONTRA _BRING_TO_STATE

El método con guion bajo usa `sudo()` y salta todas las reglas de acceso. Está pensado para automatizaciones internas del propio módulo. En cualquier comando Python, webhook o acción de servidor que escribas tú, usa siempre `bring_to_state(referencia_estado)`, que respeta los permisos del usuario que lo ejecuta.

9.7 Waypoints: puntos de restauración

Un waypoint es una versión guardada del jet a la que se puede volver. Su plantilla `cx.tower.jet.waypoint.template` define hasta cuatro flight plans, uno por cada momento del ciclo de vida:

Plan	Cuándo se ejecuta	Qué haría en efactusv
Create Flight Plan	Al crear el waypoint	Volcar la base de datos y el filestore a /opt/efactusv/waypoints/<id>/
Leave Flight Plan	Antes de abandonar el waypoint actual	Guardar el estado actual antes de saltar a otro punto
Arrive Flight Plan	Al llegar al waypoint	Restaurar el volcado y reiniciar los contenedores
Delete Flight Plan	Antes de borrar el waypoint	Eliminar los archivos del volcado

El waypoint recorre los estados *draft* → *preparing* → *ready* y, al usarse, *arriving* → *current*. Además guarda una instantánea de los valores de las variables del jet, que se restauran al volver a él.

EL CASO DE USO QUE JUSTIFICA TODO ESTO

Antes de actualizar el módulo de facturación electrónica de un cliente, un comando **Create Waypoint** hace el volcado. Si la actualización rompe algo, **Fly to** devuelve la instancia al estado previo —datos y variables incluidos— en un clic, sin restaurar backups a mano en plena llamada del cliente.

9.8 Códigos de error del subsistema de jets

Código	Significado	Causa habitual
-501	Acción de jet no encontrada	Referencia mal escrita en un comando
-502	Plantilla de jet no encontrada	Referencia inválida
-503	Jet no encontrado	El jet fue borrado o no está en contexto
-504	Error de estado del jet	No existe camino hacia el estado pedido
-505	Acción no disponible	El jet está ocupado (en estado transitorio) o la acción no aplica desde su estado actual
-506	Dependencias no satisfechas	Un jet requerido no está en el estado exigido
-507	Plantilla de waypoint no encontrada	Falta configurarla en el jet template
-508	Fallo al crear el waypoint	El jet está ocupado, o la plantilla no pertenece a ese jet template

Caso práctico: la plantilla «Instancia Odoo efactusv»

Todo lo anterior converge aquí. En este capítulo construimos, pieza a pieza, el jet template que aprovisiona una instancia Odoo completa para un cliente: contenedores, configuración, dominio, SSL y base de datos inicializada. Al terminar tendrás un botón que convierte un contrato firmado en un cliente operativo.

10.1 Inventario: lo que hay que tener antes de empezar

✓	Elemento	Capítulo
☐	Servidor de producción dado de alta y con <i>Test Connection</i> en verde	4
☐	Secretos <code>db_password</code> , <code>admin_passwd</code> , <code>ghcr_token</code>	3.4
☐	Secretos de OVH (si vas a automatizar el DNS)	3.4 / 11
☐	Variables <code>instance_name</code> , <code>instance_domain</code> , <code>odoo_image</code> , <code>odoo_http_port</code> , <code>odoo_gevent_port</code> , <code>odoo_workers</code> , <code>base_dir</code> , <code>admin_email</code>	3.5
☐	Las tres plantillas de archivo (<code>compose</code> , <code>odoo.conf</code> , <code>vhost</code>)	6.4
☐	Imagen Docker de efactusv publicada y accesible	—

10.2 Paso 1 — Crear los estados

En **Settings > Jets > States**, crea los seis estados de la tabla 9.2: `running`, `stopped`, `starting`, `stopping`, `installing` y `error`. Son globales: los reutilizarán todas tus plantillas futuras.

10.3 Paso 2 — Crear los flight plans

Antes de la plantilla, sus planes. Créalos en **Commands > Flight Plans** con **On Error = Exit with command exit code** en todos.

Plan A — `efactusv_instance_install`

El plan de aprovisionamiento completo. Es el más largo; cada línea es un comando que hay que crear previamente en **Commands > Commands**.

Seq	Comando	Acción	Contenido
10	Crear estructura	SSH Command	<code>mkdir -p {{ base_dir }}/{{ instance_name }}/config</code>
20	Subir docker-compose	Create/Update File	Plantilla <code>docker-compose.yml</code> , <i>If file exists = Overwrite</i>
30	Subir odoo.conf	Create/Update File	Plantilla <code>odoo.conf</code> , <i>Overwrite</i>
40	Autenticar en el registro	SSH Command	<code>echo "#!cxtower.secret.ghcr_token!#" docker login ghcr.io -u efactusv --password-stdin</code>
50	Levantar la pila	SSH Command	<code>cd {{ base_dir }}/{{ instance_name }} && docker compose pull && docker compose up -d</code>
60	DNS del subdominio	Run Flight Plan	Subplan <code>efactusv_instance_dns</code> (capítulo 11)
70	Certificado y vhost	Run Flight Plan	Subplan <code>efactusv_instance_ssl</code> (capítulo 11)

Seq	Comando	Acción	Contenido
80	Esperar a Odoo	SSH Command	Sondeo del puerto, ver más abajo
90	Inicializar la BD	SSH Command	Ver más abajo
99	Registrar la URL	Python Code	Guarda <code>https://{{ instance_domain }}</code> en el campo <code>URL</code> del jet

Línea 80 — esperar a que Odoo responda

Sin esta espera, la inicialización de la base de datos falla de forma intermitente porque PostgreSQL todavía no acepta conexiones. Es la causa más común de "a veces funciona".

```
for i in $(seq 1 60); do
  if docker exec db-{{ instance_name }} pg_isready -U odoo -q; then
    echo "PostgreSQL listo tras ${i}s"
    exit 0
  fi
  sleep 1
done
echo "Timeout esperando a PostgreSQL"
exit 1
```

Línea 90 — inicializar la base de datos

```
docker exec odoo-{{ instance_name }} odoo \
  -c /etc/odoo/odoo.conf \
  -d {{ instance_name }} \
  -i ll0n_sv_efactusv \
  --without-demo=all \
  --stop-after-init
```

DOS DETALLES QUE AHORRAN INCIDENCIAS

`--without-demo=all` evita que la instancia de un cliente nazca con datos de demostración, algo que en un sistema de facturación electrónica sería un problema serio. Y como el `entrypoint` del contenedor arranca Odoo, este comando lanza un **segundo** proceso dentro del mismo contenedor: correcto para inicializar, pero no lo uses para actualizaciones rutinarias sin detener antes el proceso principal.

Línea 99 — guardar la URL en el jet

```
jet = env["cx.tower.jet"].browse(jet_id)
dominio = jet.get_variable_value("instance_domain")
jet.sudo().write({"url": "https://%s" % dominio})
result = {"exit_code": 0, "message": "URL registrada: https://%s" % dominio}
```

Merece la pena: a partir de aquí `{{ tower.jet.url }}` y `{{ tower.jet.hostname }}` funcionan en cualquier comando o plantilla, y la tarjeta del jet muestra un enlace directo a la instancia del cliente.

Plan B — `efactusv_instance_stop`

```
cd {{ base_dir }}/{{ instance_name }} && docker compose stop
```

Plan C — `efactusv_instance_start`

```
cd {{ base_dir }}/{{ instance_name }} && docker compose start
```

Plan D — `efactusv_instance_uninstall`

Seq	Paso	Contenido
10	Backup final de seguridad	<code>docker exec db-{{ instance_name }} pg_dump -U odoo -Fc {{ instance_name }} > {{ base_dir }}/backups/{{ instance_name }}-FINAL-{{ tower.tools.today_underscore }}.dump</code>
20	Bajar la pila y borrar volúmenes	<code>cd {{ base_dir }}/{{ instance_name }} && docker compose down -v</code>
30	Quitar el vhost	<code>rm -f /etc/nginx/sites-enabled/{{ instance_name }}.conf && nginx -t && systemctl reload nginx</code>
40	Borrar el directorio	<code>rm -rf {{ base_dir }}/{{ instance_name }}</code>

Seq	Paso	Contenido
50	Retirar el registro DNS	Comando Python contra la API de OVH

EL PASO 10 NO ES OPCIONAL

`docker compose down -v` borra los volúmenes: los datos del cliente desaparecen de forma irreversible. Haz siempre el volcado final **antes**, y con *On Error = Exit* para que un fallo del backup detenga la destrucción. Un cliente que vuelve dos meses después pidiendo sus datos es un escenario real; el volcado final es lo que te salva.

Plan E — `efactusv_server_prepare` (instalación de la plantilla)

Este no se ejecuta por cliente sino **una vez por servidor**, cuando instalas la plantilla en él:

Seq	Paso	Contenido
10	Verificar Docker	<code>docker --version && docker compose version</code>
20	Verificar nginx y certbot	<code>nginx -v && certbot --version</code>
30	Crear la raíz de despliegue	<code>mkdir -p {{ base_dir }}/backups && chown -R \$USER:\$USER {{ base_dir }}</code>
40	Raíz del reto ACME	<code>sudo mkdir -p /var/www/certbot</code>
50	Red Docker común	<code>docker network create efactusv_shared true</code>

10.4 Paso 3 — Crear el Jet Template

Cetmix Tower > Jets > Templates > Nuevo.

Campo	Valor
Name	Instancia Odoo efactusv
Reference	<code>efactusv_odoo_instance</code>
Icon	El logotipo de efactusv
Installation Flight Plan	<code>efactusv_server_prepare</code>
Uninstallation Flight Plan	Plan de limpieza del servidor (opcional)
Limit per server	10 — ajústalo a la capacidad real de tu VPS
Show in create wizard	Marcado
Access Level	Manager

Pestaña Variables: los valores por defecto

Variable	Valor por defecto	¿Requerida?	Nota
<code>base_dir</code>	<code>/opt/efactusv</code>	No	Igual para todos
<code>odoo_image</code>	<code>ghcr.io/porti1876/efactusv-odoo:18.0</code>	No	Se sobrescribe por jet si un cliente va en otra versión
<code>odoo_workers</code>	2	No	Variable de tipo <i>Options</i> según el plan contratado
<code>admin_email</code>	<code>admin@efactusv.com</code>	No	Para Let's Encrypt
<code>instance_name</code>	—	Sí	Identidad del cliente
<code>instance_domain</code>	—	Sí	Su subdominio
<code>odoo_http_port</code>	—	Sí	Único en el servidor
<code>odoo_gevent_port</code>	—	Sí	Único en el servidor

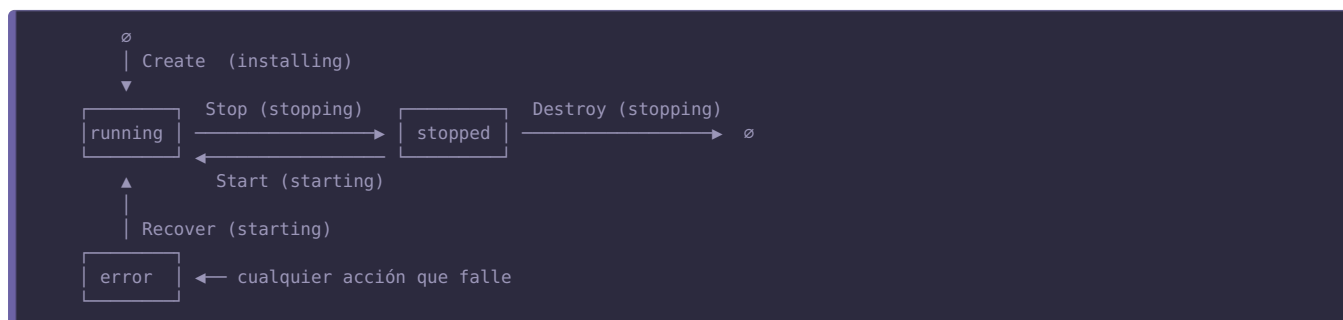
Pestaña Files

Añade las tres plantillas de archivo del capítulo 6.4. Al crearse cada jet, Tower materializa un `cx.tower.file` por cada una, ya renderizado con las variables de ese cliente.

Pestaña Actions: la máquina de estados completa

Acción	From	Transit	To	Error	Flight Plan
Create	(vacío)	installing	running	error	efactusv_instance_install
Stop	running	stopping	stopped	error	efactusv_instance_stop
Start	stopped	starting	running	error	efactusv_instance_start
Recover	error	starting	running	error	efactusv_instance_start
Destroy	stopped	stopping	(vacío)	error	efactusv_instance_uninstall

El grafo resultante:



POR QUÉ "DESTROY" PARTE DE STOPPED Y NO DE RUNNING

Obliga a un paso intermedio deliberado antes de una operación irreversible. Y como Tower calcula caminos automáticamente, "llevar el jet a ∅" desde `running` sigue funcionando: ejecuta `Stop` y luego `Destroy`. Ganas la protección sin perder la comodidad.

Pestaña Scheduled Tasks: el backup diario

Crea en `Settings > Automation > Scheduled Tasks` una tarea con:

Campo	Valor
Action	Command
Interval	1 Days
Command	Un comando SSH con el volcado (abajo)
Jet Templates	efactusv_odoo_instance — así todos los jets la heredan

```
set -e
mkdir -p {{ base_dir }}/backups
docker exec db-{{ instance_name }} pg_dump -U odoo -Fc {{ instance_name }} \
> {{ base_dir }}/backups/{{ instance_name }}-{{ tower.tools.today_underscore }}.dump
docker run --rm \
-v {{ base_dir }}/{{ instance_name }}_odoo-web-data:/data:ro \
-v {{ base_dir }}/backups:/backup \
alpine tar czf /backup/{{ instance_name }}-filestore-{{ tower.tools.today_underscore }}.tar.gz -C /data .
find {{ base_dir }}/backups -name "{{ instance_name }}-*" -mtime +14 -delete
```

UN BACKUP LOCAL NO ES UN BACKUP

Este guion deja las copias en el mismo disco que los datos. Añade un paso que las envíe fuera —OVH Object Storage, S3, otro servidor— o un fallo de disco se llevará por delante los datos y sus copias a la vez. Y prueba la restauración: un backup que nunca se ha restaurado es una hipótesis, no un respaldo.

10.5 Paso 4 — Instalar la plantilla en el servidor

Abre la plantilla, pulsa `Install on servers` y elige tu VPS de producción. Se ejecuta `efactusv_server_prepare`. Verifica en `Logs > Jet Template Installations` que el estado quedó en **Done** antes de continuar.

10.6 Paso 5 — Lanzar la primera instancia

1 Abre el asistente

Cetmix Tower > Jets > Launch New Jet

2 Rellena los datos del cliente

Campo	Valor de ejemplo
Jet Template	Instancia Odoo efactusv
Server	PROD-SV-01
Name	Ferretería El Tornillo
Partner	El cliente en Odoo
instance_name	eltornillo
instance_domain	eltornillo.efactusv.com
odoo_http_port	10069
odoo_gevent_port	10072
odoo_workers	2

3 Dispara la acción Create

El jet nace sin estado. Pulsa **Action** y elige **Create** (o **State** → *running*, que llega al mismo sitio). El jet pasa a *installing* y arranca `efactusv_instance_install`.

4 Sigue la ejecución

En `Logs > Flight Plan Log` ves el avance línea a línea. Con `cetmix_tower_server_queue` instalado, todo esto ocurre en segundo plano y la interfaz sigue respondiendo.

5 Verifica

- El jet quedó en estado `running` con su tarjeta en verde
- `https://eltornillo.efactusv.com` muestra el login de Odoo con certificado válido
- La instancia solo ofrece su propia base de datos (efecto `dedbfilter` y `list_db`)
- En el servidor: `docker ps` muestra `odoo-eltornillo` y `db-eltornillo`

10.7 Asignación automática de puertos

Pedir el puerto a mano en cada alta es la primera fuente de colisiones. Añade este comando Python como línea 5 del plan de instalación y elimina los dos campos de puerto del asistente:

```
jet = env["cx.tower.jet"].browse(jet_id)
servidor = jet.server_id

# Puertos ya usados por otros jets del mismo servidor
usados = set()
for otro in servidor.jet_ids.filtered(lambda j: j.id != jet.id):
    valor = otro.get_variable_value("odoo_http_port")
    if valor:
        usados.add(int(valor))

http = 10069
while http in usados:
    http += 100

env["cx.tower.variable.value"].sudo().create([
    {"jet_id": jet.id,
     "variable_id": env["cx.tower.variable"].search(
         [("reference", "=", "odoo_http_port")], limit=1).id,
     "value_char": str(http)},
    {"jet_id": jet.id,
     "variable_id": env["cx.tower.variable"].search(
         [("reference", "=", "odoo_gevent_port")], limit=1).id,
     "value_char": str(http + 3)},
])
result = {"exit_code": 0, "message": "Puertos asignados: %s / %s" % (http, http + 3)}
```

EL ESQUEMA DE NUMERACIÓN

Saltar de 100 en 100 (10069, 10169, 10269...) y usar `http + 3` para el websocket replica el desfase natural de Odoo entre 8069 y 8072. Con un solo vistazo a un puerto sabes de qué instancia es y cuál es su par.

Dominio propio del cliente y SSL

Toda instancia nace en un subdominio tuyo (cliente.efactusv.com), pero antes o después el cliente querrá facturacion.suempresa.com. Este capítulo automatiza ambos casos: el subdominio en el alta, y el dominio propio como operación posterior.

11.1 Configurar el acceso a la API de OVHcloud

Sigue la documentación oficial de OVH para crear una aplicación y generar las tres credenciales, y guárdalas como secretos (capítulo 3.4) con las referencias `ovh_application_key`, `ovh_application_secret` y `ovh_consumer_key`. Crea además la variable `ovh_endpoint` de tipo String con el valor de tu región: `ovh-eu`, `ovh-ca` o `ovh-us`.

PERMISOS DE LA APLICACIÓN OVH

Concede solo lo necesario: GET/POST/PUT/DELETE sobre `/domain/zone/*`. Una credencial con permisos sobre toda tu cuenta, guardada en una base de datos de Odoo, es un riesgo que no necesitas correr.

11.2 El subplan `efactusv_instance_dns`

Crea un comando de tipo **Python Code** llamado *Crear registro A en OVH*, con nivel de acceso Manager, la variable `ovh_endpoint` y los tres secretos declarados:

```
result = {"exit_code": 0, "message": None}

client = ovh.Client(
    endpoint={{ ovh_endpoint }},
    application_key=!cxtower.secret.ovh_application_key!#,
    application_secret=!cxtower.secret.ovh_application_secret!#,
    consumer_key=!cxtower.secret.ovh_consumer_key!#,
)

zona      = "efactusv.com"
subdominio = "{{ instance_name }}"
ip        = "{{ tower.server.ipv4 }}"

try:
    # ¿ya existe el registro?
    existentes = client.get(
        "/domain/zone/" + zona + "/record",
        fieldType="A", subDomain=subdominio,
    )
    if existentes:
        client.put(
            "/domain/zone/" + zona + "/record/" + str(existentes[0]),
            target=ip, ttl=300,
        )
        accion = "actualizado"
    else:
        client.post(
            "/domain/zone/" + zona + "/record",
            fieldType="A", subDomain=subdominio, target=ip, ttl=300,
        )
        accion = "creado"

    client.post("/domain/zone/" + zona + "/refresh")
    result["message"] = "Registro A %s.%s %s -> %s" % (subdominio, zona, accion, ip)
except Exception as e:
    result["exit_code"] = 1
    result["message"] = "Error OVH: " + str(e)
```

IDEMPOTENCIA

El comando comprueba si el registro existe y lo actualiza en lugar de duplicarlo. Un plan de aprovisionamiento se reintenta más veces de las que uno espera; que cada paso pueda ejecutarse dos veces sin romper nada es lo que hace que reintentar sea seguro.

Esperar la propagación

Segunda línea del subplan, esta vez SSH: sin ella, certbot fallará el reto ACME porque el nombre todavía no resuelve.

```
for i in $(seq 1 30); do
  if getent hosts {{ instance_domain }} | grep -q "{{ tower.server.ipv4 }}"; then
    echo "DNS propagado tras $((i*10))s"
    exit 0
  fi
  sleep 10
done
echo "Timeout esperando la propagación DNS"
exit 1
```

11.3 El subplan `efactusv_instance_ssl`

Seq	Paso	Contenido
10	vhost temporal solo HTTP	Publica un bloque <code>listen 80</code> con el <code>location /.well-known/acme-challenge/</code> , sin referencias a certificados
20	Recargar nginx	<code>sudo nginx -t && sudo systemctl reload nginx</code>
30	Emitir el certificado	Ver abajo
40	vhost definitivo	Plantilla completa del capítulo 6.4, con los bloques 80 y 443
50	Recargar nginx	<code>sudo nginx -t && sudo systemctl reload nginx</code>

```
sudo certbot certonly --webroot -w /var/www/certbot \
-d {{ instance_domain }} \
--non-interactive --agree-tos -m {{ admin_email }} \
--keep-until-expiring
```

Opción	Por qué
<code>--webroot -w /var/www/certbot</code>	Reto HTTP-01 sin tocar la configuración de nginx: más predecible que <code>--nginx</code> cuando el vhost lo gestiona Tower
<code>--non-interactive --agree-tos</code>	Sin ellas, certbot se queda esperando entrada y el comando expira
<code>--keep-until-expiring</code>	Hace el paso idempotente: si el certificado sigue vigente, no vuelve a pedirlo. Clave para no chocar con los límites de emisión de Let's Encrypt al reintentar.

LOS LÍMITES DE LET'S ENCRYPT SON REALES

Cinco fallos de validación por cuenta, dominio y hora; cincuenta certificados por dominio registrado y semana. Depurar un plan de aprovisionamiento sin `--keep-until-expiring`, o sin usar el entorno de pruebas `--dry-run`, te dejará bloqueado durante horas. Depura contra staging y pasa a producción cuando el plan funcione de principio a fin.

Renovación automática

El paquete de certbot instala su propio temporizador de renovación, pero necesita recargar nginx al renovar. Añádelo al plan `efactusv_server_prepare`:

```
echo '#!/bin/sh
systemctl reload nginx' | sudo tee /etc/letsencrypt/renewal-hooks/deploy/reload-nginx.sh
sudo chmod +x /etc/letsencrypt/renewal-hooks/deploy/reload-nginx.sh
```

11.4 El dominio propio del cliente

Este es un proceso posterior al alta, normalmente a petición del cliente. La diferencia crítica **el DNS no lo controlas tú**, sino el cliente en su propio registrador.

Modelo de datos

Crea una variable `custom_domain` de tipo String, vacía por defecto. Solo los jets de clientes con dominio propio la tendrán rellena.

Instrucciones para el cliente

LO QUE DEBES PEDIRLE AL CLIENTE

Que cree en su proveedor de DNS un registro **CNAME**:

```
facturacion.suempresa.com. CNAME eltornoillo.efactusv.com.
```

El CNAME es preferible a un registro A: si algún día cambias el servidor de sitio o su IP, el dominio del cliente sigue funcionando sin que tenga que tocar nada.

El plan `efactusv_instance_custom_domain`

Seq	Paso	Contenido
10	Verificar el CNAME	Comprueba que el dominio resuelve a la IP del servidor; si no, aborta con un mensaje claro para soporte
20	vhost adicional HTTP	Segundo bloque <code>server</code> con el nombre del cliente y el <code>location</code> del reto ACME
30	Recargar nginx	<code>sudo nginx -t && sudo systemctl reload nginx</code>
40	Ampliar el certificado	Un solo certificado para ambos nombres (abajo)
50	vhost definitivo con los dos nombres	<code>server_name {{ instance_domain }} {{ custom_domain }};</code>
60	Fijar la URL base en Odoo	Ver abajo
70	Recargar nginx	<code>sudo nginx -t && sudo systemctl reload nginx</code>

Paso 10 — verificación del CNAME

```
esperado="{{ tower.server.ipv4 }}"
obtenido=$(getent hosts {{ custom_domain }} | awk '{print $1}' | head -1)
if [ "$obtenido" != "$esperado" ]; then
    echo "El dominio {{ custom_domain }} resuelve a '$obtenido' y se esperaba '$esperado'."
    echo "El cliente aún no ha creado el CNAME, o no ha propagado."
    exit 1
fi
echo "CNAME verificado correctamente"
```

Paso 40 — un certificado para los dos nombres

```
sudo certbot certonly --webroot -w /var/www/certbot \
--cert-name {{ instance_domain }} \
-d {{ instance_domain }} -d {{ custom_domain }} \
--expand --non-interactive --agree-tos -m {{ admin_email }}
```

`--cert-name` mantiene la ruta de los certificados (`/etc/letsencrypt/live/<instance_domain>/`) estable, para que el vhost no cambie; `--expand` añade el nuevo nombre al certificado existente en lugar de crear uno paralelo.

Paso 60 — decirle a Odoo cuál es su URL

Sin esto, los enlaces de los correos, los códigos QR de las facturas y los portales seguirán apuntando al subdominio de `efactusv` en lugar de al dominio del cliente:

```
docker exec odoo-{{ instance_name }} odoo shell \
-c /etc/odoo/odoo.conf -d {{ instance_name }} <<'PYEOF'
env["ir.config_parameter"].sudo().set_param(
    "web.base.url", "https://{{ custom_domain }}")
env["ir.config_parameter"].sudo().set_param("web.base.url.freeze", "True")
env.cr.commit()
PYEOF
```

WEB.BASE.URL.FREEZE

Sin ese segundo parámetro, Odoo reescribe `web.base.url` con el host de la primera petición de un administrador. Si un técnico entra por el subdominio interno, la URL base vuelve a cambiar y los correos del cliente salen con el dominio equivocado. Congélala.

Convertirlo en una acción de jet

Para que soporte no tenga que ejecutar planes a mano, añade a la plantilla una acción más:

Acción	From	Transit	To	Flight Plan
Add custom domain	running	starting	running	efactusv_instance_custom_domain

Es una transición que empieza y acaba en `running`: el jet no cambia de estado, pero pasa por el estado transitorio mientras trabaja, lo que impide ejecuciones simultáneas y muestra el progreso en la interfaz. Basta con rellenar la variable `custom_domain` del jet y pulsar la acción.

Contratos: enlazar el negocio con la infraestructura

Hasta aquí, crear la instancia de un cliente sigue siendo una tarea técnica. El módulo `efactusv_tower_contract` la traslada al contrato: quien firma el alta la aprovisiona, y cuando el contrato termina la instancia se detiene sola. Es la pieza que convierte esto en un negocio operable.

12.1 Qué añade el módulo

Dónde	Elemento	Función
Contrato <code>contract.contract</code>	Instance Template	Qué jet template se usará para las instancias de este contrato
	Instance Server	En qué servidor se crearán
	Create Instance	Botón que crea el jet enlazado al contrato y a su cliente
	Botón <i>Instances</i>	Contador con acceso directo a las instancias del contrato
Jet <code>cx.tower.jet</code>	Contract	Contrato con el que se factura la instancia. Con seguimiento en el chatter.
	Acción <i>Ver contrato</i>	Navegación en sentido inverso
Jet Template	State on Contract Termination	Estado al que se llevan las instancias cuando el contrato se termina
	State on Contract Reactivation	Estado al que vuelven si se cancela la terminación

12.2 Configuración, una sola vez

1 Configura los estados del ciclo de vida en la plantilla

Abre `Jets > Templates` → *Instancia Odoo efactusv* y rellena:

Campo	Valor
State on Contract Termination	<code>stopped</code>
State on Contract Reactivation	<code>running</code>

Si dejas alguno vacío, ese evento no hace nada: la instancia sigue como esté. Es la configuración correcta si prefieres decidir caso por caso.

2 Prepara el contrato del cliente

En el contrato, pestaña de información general, sección **SaaS Instances**:

- **Instance Template:** *Instancia Odoo efactusv*
- **Instance Server:** el VPS de producción con capacidad disponible

12.3 El alta de un cliente, de principio a fin

1. Comercial cierra la venta
 - ▼
2. Se crea el contrato en Odoo (cliente, plan, periodicidad de facturación)
 - ▼
3. En el contrato: Instance Template + Instance Server
 - ▼
4. Botón "Create Instance"
 - ├ valida que ambos campos estén rellenos
 - ├ valida que el contrato no esté terminado
 - ├ crea el jet: nombre = código del contrato, partner y contract enlazados
 - ├ escribe una nota en el chatter del contrato
 - └ abre la ficha del jet recién creado
 - ▼
5. En el jet: rellenar las variables del cliente y disparar la acción "Create"
 - ▼
6. Aprovisionamiento automático (capítulo 10)
 - ▼
7. Se entregan las credenciales; el contrato empieza a facturar

COMPROBACIONES DEL BOTÓN

Create Instance se niega a funcionar si falta la plantilla o el servidor, y también si el contrato ya está terminado. Si el jet no se crea porque el servidor alcanzó su *Limit per server*, el mensaje de error te lo dice explícitamente: hay que elegir otro servidor o subir el límite.

12.4 Terminación y reactivación automáticas

Al terminar el contrato

Cuando alguien usa el asistente de terminación de contrato (impago, baja voluntaria, fin de vigencia), el módulo intercepta la operación y, para cada instancia activa del contrato, la lleva al estado configurado. Con la configuración recomendada, la instancia pasa a `stopped`: **los contenedores se paran pero los datos siguen intactos**.

Al cancelar la terminación

Si el cliente paga y se cancela la terminación, las instancias vuelven a `running` ejecutando el plan de arranque. La recuperación es de segundos, no de horas.

La regla de oro del diseño: los fallos no bloquean

UN FALLO DE INFRAESTRUCTURA NUNCA BLOQUEA AL EQUIPO DE ADMINISTRACIÓN

Si una instancia no puede detenerse —el servidor está caído, el plan falla—, el módulo **no** lanza una excepción. Registra el error en el log técnico y publica un mensaje en el chatter del contrato: «*No se pudo llevar la instancia X al estado Y. Revise los logs en Cetmix Tower.*» La terminación del contrato se completa igualmente.

Es una decisión deliberada: la persona de administración no puede hacer nada respecto a un servidor caído, y bloquear su trabajo no arregla nada. El aviso queda visible para quien sí puede actuar.

Consecuencia operativa

Como los fallos son silenciosos para el flujo del contrato, **alguien tiene que vigilarlos**. Dos medidas concretas:

- Añade el equipo de operaciones como seguidor de los contratos con instancia, para que los mensajes del chatter les lleguen.
- Crea un filtro guardado en la lista de jets: `estado = error`, y revísalo a diario. Es tu bandeja de incidencias de infraestructura.

12.5 Lo que este módulo deliberadamente no hace

No hace	Cómo resolverlo
Crear la instancia automáticamente al confirmar el contrato	Añade una acción de servidor sobre <code>contract.contract</code> que llame a <code>action_create_tower_jet()</code> . Se dejó manual a propósito: aprovisionar cuesta dinero y conviene una confirmación humana.

No hace	Cómo resolverlo
Rellenar las variables del jet desde el contrato	Extiende <code>action_create_tower_jet</code> para pasar <code>variable_values</code> derivados del contrato: <code>instance_name</code> desde el código del contrato, <code>odoo_workers</code> desde el producto contratado.
Destruir la instancia al terminar el contrato	Intencionadamente. Detener conserva los datos; destruir los borra. Haz la destrucción una decisión manual, tras un periodo de retención de 30-90 días.
Facturar por consumo	Enlaza las líneas del contrato con las variables del jet (número de usuarios, workers) mediante una acción planificada.

Addons y proyectos Git

Una instancia SaaS de facturación electrónica es su código: `l10n_sv_efactusv`, los módulos OCA de los que depende y tus personalizaciones. El módulo `cetmix_tower_git` modela ese árbol de fuentes dentro de Odoo y genera el `repos.yaml` que `git-aggregator` necesita para reconstruirlo.

13.1 Jerarquía de objetos

Objeto	Qué representa
Git Repo	Un repositorio: URL, proveedor (GitHub, GitLab, Bitbucket...), propietario, si es privado y qué secreto lo autentica
Git Remote	Un remoto concreto de un repo dentro de una fuente: protocolo (HTTPS/SSH/git) y <i>head</i> (rama, etiqueta o commit)
Git Source	Un directorio destino en el servidor, alimentado por uno o varios remotos. Es una entrada del <code>repos.yaml</code> .
Git Project	El conjunto completo de fuentes que compone el árbol de addons de un servidor

The screenshot shows the 'Demo Git Project' configuration page in Cetmix Tower. The page has a sidebar with 'Edit' and 'Create' buttons. The main content area shows the project name 'demo_git_project' and a list of sources. The sources table has columns for Name, Remotes, and Enabled. The sources are color-coded: blue for private and orange for semi-private.

Name	Remotes	Enabled
Cetmix Tower	2	☒
OCA Web	1	☒
Odoo Enterprise (Private)	1	☒
Sample Private Bitbucket	2	☒
Sample Semi Private Gitlab	2	☒

* Sources where all remotes are private
* Sources where some remotes are private

Figura 13.1 — Ficha de un proyecto Git en `Tools > Git Projects`. Cada fuente muestra su número de remotos y su interruptor de activación. El código de color de los nombres distingue las fuentes con remotos privados (azul) de las parcialmente privadas (naranja): una lectura inmediata de dónde harán falta credenciales.

13.2 Montar el proyecto de addons de efactusv

1 Da de alta los repositorios

Repositorio	¿Privado?	Contenido
odoo/odoo	No	Núcleo de Odoo 18
OCA/contract	No	Contratos y facturación recurrente
OCA/server-tools	No	Utilidades de servidor
efactusv/efactusv	Sí	Vuestros módulos: <code>l10n_sv_efactusv</code> , integraciones DTE

Para los privados, crea un secreto con el token de lectura y selecciónalo en el campo **Secret** del repositorio.

2 Crea el proyecto y sus fuentes

Una fuente por directorio destino. Si una fuente tiene varios remotos, git-aggregator los fusiona: así se aplican parches propios sobre un módulo OCA sin bifurcar el repositorio entero.

3 Asocia el proyecto al servidor

En la pestaña **Git Projects** de la ficha del servidor, o en el campo **Servers** del proyecto.

4 Genera el repos.yaml

El módulo produce el archivo a partir del proyecto, respetando el campo **Git aggregator root dir**. Ese archivo se convierte en una plantilla de archivo (capítulo 6) que el flight plan deposita en el servidor.

13.3 Dos estrategias de despliegue de addons

Estrategia	Cómo funciona	Cuándo
Imagen inmutable recomendada	El código se hornea en la imagen Docker en el CI. Desplegar es cambiar la variable <code>odoo_image</code> y hacer <code>pull + up -d</code> .	Producción. Todos los clientes con la misma etiqueta corren exactamente el mismo código, y volver atrás es cambiar una etiqueta.
Árbol en el servidor	El servidor guarda un checkout gestionado por git-aggregator, montado como volumen en el contenedor. Desplegar es <code>gitaggregate + reiniciar</code> .	Desarrollo y staging, o cuando un cliente necesita una rama propia temporal.

LA RECOMENDACIÓN PARA EFACTUSV

Usa imágenes inmutables en producción y reserva el modelo Git para staging. Una plataforma de facturación electrónica tiene requisitos de trazabilidad: poder afirmar "el cliente X corría exactamente la imagen 18.0.4.2 el día de esa factura" vale mucho más que la flexibilidad de tocar el código en caliente. El módulo Git sigue siendo útil: es donde declaras, de forma versionada, qué compone cada imagen.

13.4 Un plan de actualización seguro

Seq	Paso	Detalle
10	Crear waypoint	Comando <i>Create Waypoint</i> : volcado de BD y filestore antes de tocar nada
20	Detener la instancia	<code>docker compose stop odoo</code> — la base sigue viva para la migración
30	Actualizar la imagen	<code>docker compose pull && docker compose up -d --no-start odoo</code>
40	Actualizar los módulos	<code>docker compose run --rm odoo odoo -c /etc/odoo/odoo.conf -d {{ instance_name }} -u all --stop-after-init</code>
50	Arrancar	<code>docker compose up -d</code>
60	Prueba de humo	<code>curl -fsS https://{{ instance_domain }}/web/login > /dev/null</code>

Si el paso 60 falla, el waypoint del paso 10 permite volver atrás con un clic.

Automatización: webhooks, tareas programadas y cola

14.1 Ejecución asíncrona con `cetmix_tower_server_queue`

Sin este módulo, un flight plan de aprovisionamiento de cinco minutos mantiene ocupado un worker HTTP de Odoo durante cinco minutos. Con dos o tres altas simultáneas, la instancia madre deja de responder.

Con él, los comandos y flight plans se encolan como trabajos de `queue_job`: la interfaz responde de inmediato, el trabajo se ejecuta en segundo plano y las notificaciones llegan por el bus cuando termina. Los reintentos y el seguimiento están en `Ajustes > Técnico > Queue Jobs`.

DIMENSIONA LOS CANALES

Con `channels = root:2` solo se ejecutan dos trabajos a la vez. Si aprovisionas varios clientes en paralelo, sube a `root:4` y considera un canal dedicado para las tareas largas, de modo que un aprovisionamiento no deje esperando a todos los backups nocturnos.

14.2 Tareas programadas

En `Settings > Automation > Scheduled Tasks`. Ejecutan un comando o un flight plan sobre un conjunto de destinos.

Campo	Detalle
Action	Command o Flight Plan
Destinos	Servidores, plantillas de servidor, jets o jet templates . Esta última es la más potente: la tarea se aplica a todos los jets existentes y futuros de esa plantilla.
Interval	Minutos, horas, días, semanas, meses o días de la semana
Días de la semana	Con <code>interval type = Days of Week</code> , marcas los días concretos: backups completos solo los domingos
Next call / Last call	Próxima y última ejecución, calculadas por el sistema
Custom Variable Values	Variables específicas para esta ejecución

El disparador real es el cron `Run scheduled tasks` de Odoo. Si va cada 5 minutos, esa es tu granularidad máxima: una tarea "cada minuto" no se ejecutará cada minuto.

Calendario sugerido para efactusv

Tarea	Frecuencia	Destino
Backup de BD y filestore	Diaria, 02:00	Jet template <i>Instancia Odoo efactusv</i>
Purga de backups antiguos	Semanal	Servidores de producción
Comprobación de espacio en disco	Cada 6 horas	Servidores de producción
Prueba de humo HTTP de cada instancia	Cada 15 min	Jet template
Renovación de certificados	Diaria	Servidores (verificación; certbot ya renueva solo)
<code>docker system prune -af --filter until=168h</code>	Semanal	Servidores de producción

14.3 Webhooks entrantes

Requiere pertenecer al grupo Root. Un webhook es un endpoint HTTP que ejecuta código Python de Tower cuando alguien lo llama. Es la puerta para que un sistema externo dispare un aprovisionamiento.

Paso 1 — El autenticador

Settings > Automation > Webhook Authenticators . Su código decide si la petición se acepta. Dispone de `headers`, `raw_data` y `payload`, y debe devolver:

```
result = {"allowed": <bool>, "http_code": <int, opcional>, "message": <str, opcional>}
```

Ejemplo con firma HMAC, que es como debería hacerse en producción:

```
import hmac, hashlib

secreto = "#!cxtower.secret.webhook_signing_key!#"
firma_recibida = headers.get("X-Signature-256", "")
firma_esperada = "sha256=" + hmac.new(
    secreto.encode(), raw_data.encode(), hashlib.sha256
).hexdigest()

if hmac.compare_digest(firma_recibida, firma_esperada):
    result = {"allowed": True}
else:
    result = {"allowed": False, "http_code": 403, "message": "Firma inválida"}
```

Paso 2 — El webhook

Settings > Automation > Webhooks . La URL final es `<tu_url_de_tower>/cetmix_tower_webhooks/<endpoint>`.

Campo	Detalle
Enabled	Desmárcalo para desactivarlo sin borrarlo
Authenticator	El del paso anterior
Endpoint	El segmento final de la URL
Run as User	Usuario bajo cuyos permisos se ejecuta el código
Code	El procesamiento; devuelve <code>{"exit_code": int, "message": str}</code>

RUN AS USER MERECE UN MINUTO DE REFLEXIÓN

El código del webhook corre con todos los permisos de ese usuario, y quien invoca el endpoint es cualquiera que supere el autenticador. No pongas ahí un administrador por comodidad: crea un usuario técnico específico, con el grupo de Tower mínimo necesario y sin acceso a contabilidad ni a datos de clientes.

Ejemplo: alta automática desde el sitio web

```

result = {"exit_code": 0, "message": None}

nombre = (payload.get("instance_name") or "").strip().lower()
correo = payload.get("email")
if not nombre or not correo:
    result = {"exit_code": 1, "message": "Faltan instance_name o email"}
else:
    partner = env["res.partner"].sudo().search([("email", "=", correo)], limit=1)
    if not partner:
        partner = env["res.partner"].sudo().create({
            "name": payload.get("company_name") or correo, "email": correo,
        })

    plantilla = env["cx.tower.jet.template"].sudo().search(
        [("reference", "=", "efactusv_odoo_instance")], limit=1)

    # Elige el servidor con menos jets entre los que tienen la plantilla instalada
    servidor = min(plantilla.server_ids, key=lambda s: len(s.jet_ids), default=False)

    if not servidor:
        result = {"exit_code": 2, "message": "No hay servidor disponible"}
    else:
        jet = plantilla.create_jet(
            servidor, name=nombre, partner_id=partner.id,
            variable_values={
                "instance_name": nombre,
                "instance_domain": nombre + ".efactusv.com",
            },
        )
        if not jet:
            result = {"exit_code": 3, "message": "Límite de jets alcanzado"}
        else:
            jet.bring_to_state("running")
            result["message"] = "Instancia %s en aprovisionamiento" % nombre

```

Los registros de cada llamada quedan en los logs de webhook, conservados el número de días configurado en los ajustes generales.

ANTES DE EXPONER ESTO A INTERNET

Valida `instance_name` con una expresión regular estricta (`^[a-z][a-z0-9]{2,20}$`): ese valor acaba en rutas de sistema de archivos, nombres de contenedor y nombres de base de datos. Y limita la frecuencia de llamadas: sin ello, un bucle en el cliente puede aprovisionar cien instancias en un minuto.

YAML: versionar la configuración

Toda la configuración que has construido —comandos, planes, plantillas de archivo, jet templates— vive en la base de datos. El módulo `cetmix_tower_yaml` la convierte en texto: apto para Git, para revisión por pares y para promover cambios de desarrollo a producción sin repetirlos a mano.

15.1 Exportar

Los objetos exportables ganan una pestaña **YAML** en su formulario. El usuario necesita tener marcada la casilla **Export** de la sección *Cetmix Tower YAML* de su ficha (Figura 3.1).

Se puede exportar	Notas
Comandos, flight plans y sus líneas y acciones	Incluye el código completo
Plantillas de archivo	Con su contenido
Plantillas de servidor y servidores	Sin credenciales
Jet templates, estados, acciones, waypoints, dependencias	La máquina de estados completa
Variables, opciones y valores	—
Etiquetas, sistemas operativos, tareas programadas, accesos directos	—
Llaves y secretos	Solo la referencia, nunca el valor

LOS SECRETOS NO VIAJAN, Y ESTÁ BIEN ASÍ

El YAML exporta la *referencia* del secreto, no su contenido. Al importar en otro entorno debes crear allí un secreto con la misma referencia y su propio valor. Es exactamente el comportamiento correcto: el YAML es apto para Git, y las credenciales de producción no deben estar en Git.

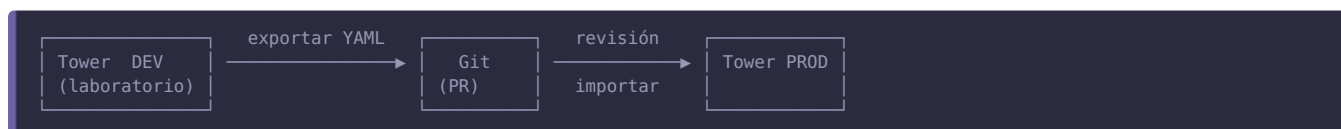
15.2 Importar

`Cetmix Tower > Tools > Import YAML`. Requiere la casilla **Import** en la ficha del usuario. El emparejamiento se hace por **referencia**: si ya existe un objeto con esa referencia, se actualiza; si no, se crea.

LAS REFERENCIAS SON LA IDENTIDAD

Por eso esta guía insiste tanto en fijar referencias estables y no cambiarlas. Cambiar la referencia de un comando después de haberlo exportado hace que la siguiente importación cree un duplicado en lugar de actualizar el original, y tu producción acaba con dos comandos casi idénticos y un plan apuntando al equivocado.

15.3 Un flujo de trabajo que funciona



1. Toda configuración nueva se construye y se prueba en una instancia de laboratorio.
2. Se exporta a YAML y se sube al repositorio en una rama.
3. El pull request permite revisar el cambio: se ve exactamente qué comando cambió y en qué línea.
4. Tras aprobarlo, se importa en producción.

Guarda los YAML junto al código, por ejemplo en `tower/config/` del repositorio de despliegue, con un archivo por dominio funcional: `commands.yaml`, `plans.yaml`, `jet_templates.yaml`, `file_templates.yaml`.

EL BENEFICIO QUE NO SE VE HASTA QUE HACE FALTA

El día que un flight plan de producción deja de funcionar, `git log` sobre el YAML te dice quién lo cambió, cuándo y qué línea tocó. Sin esto, el mismo diagnóstico son horas de comparar campos a ojo entre dos entornos.

Operación diaria y diagnóstico

Una plataforma que se aprovisiona sola sigue necesitando que alguien la mire. Este capítulo reúne las rutinas, los síntomas más frecuentes y su causa real.

16.1 Rutinas

Cada día

- Revisar la lista de jets filtrada por **estado = error**. Debería estar vacía.
- Revisar `Logs > Flight Plan Log` filtrando por ejecuciones fallidas de las últimas 24 horas.
- Comprobar que los backups nocturnos terminaron: la tarea programada muestra su *Last call*.
- Ojea `Ajustes > Técnico > Queue Jobs` en busca de trabajos fallidos.

Cada semana

- Espacio en disco de cada servidor de producción.
- Certificados que caducan en menos de 20 días (`certbot certificates`).
- Densidad de jets por servidor frente al *Limit per server*: ¿hace falta otro VPS?
- Probar la restauración de un backup en un entorno aparte. Uno al azar basta.

Cada mes

- Exportar la configuración a YAML y confirmar que el repositorio está al día.
- Revisar accesos: quién tiene nivel Root y si sigue necesitándolo.
- Comprobar que los jets sin contrato activo son los que esperas.

16.2 Diagnóstico por síntoma

Conexión

Síntoma	Causa probable	Solución
<i>Host key not found for server...</i>	Falta la huella del servidor	Acción Show host key , o marcar <i>Skip host key</i> solo en laboratorio
Fallo de autenticación	Clave pública no autorizada, o <i>SSH Auth Mode</i> incoherente	Verificar <code>authorized_keys</code> y que el modo coincida con la credencial cargada
Conecta pero fallan los archivos	Sin permiso de escritura o sin subsistema SFTP	Revisar permisos del directorio y <code>Subsystem sftp</code> en <code>sshd_config</code>
Todo expira	Cortafuegos o puerto SSH distinto	Probar <code>ssh -p PUERTO usuario@ip</code> desde el servidor de Odoo

Comandos y planes

Síntoma	Causa probable	Solución
El comando no aparece en la lista	Nivel de acceso superior al del usuario, o restringido a otros servidores/SO	Revisar <i>Access Level</i> , <i>Servers</i> y <i>Operating Systems</i> del comando
Código <code>-201</code> o <code>-301</code>	Ya hay una ejecución en curso	Esperar, o activar <i>Allow Parallel Run</i> si es seguro
Código <code>-206</code>	Se superó el <i>Command Timeout</i>	Subir el timeout, o partir el comando en pasos más cortos
Código <code>-207</code>	SO incompatible	Ajustar el campo <i>Operating Systems</i> o el SO del servidor

Síntoma	Causa probable	Solución
Las variables salen sin sustituir	La variable no tiene valor en ningún nivel de la cascada	Usar la pestaña <i>Preview</i> antes de ejecutar; comprobar el valor en el jet, el servidor y el global
El comando Python no ve la librería	El módulo del proveedor no está instalado	Instalar <code>cetmix_tower_ovh</code> o <code>_aws</code>
La interfaz se queda "pensando"	Websocket no proxeado	Añadir el <code>location /websocket</code> a nginx (capítulo 2.4)

Jets

Síntoma	Causa probable	Solución
<i>No path found to bring the jet...</i>	Falta una acción que una ambos estados	Revisar el grafo: cada estado estable necesita camino de ida y de vuelta
Código <code>-505</code>	El jet está en estado transitorio (ocupado)	Esperar a que termine; si quedó colgado, ver abajo
El jet se quedó en <code>installing</code>	El plan falló sin llegar a fijar el estado final	Revisar el log del plan, corregir la causa y llevarlo manualmente a <code>error</code> o <code>running</code>
<code>create_jet</code> devuelve <code>False</code>	Se alcanzó el <i>Limit per server</i>	Otro servidor, o subir el límite si hay capacidad real
Código <code>-506</code>	Dependencias no satisfechas	Revisar Logs > Jet Requests y el estado del jet requerido
No se puede desinstalar la plantilla	Quedan jets, o hay plantillas dependientes	Destruir los jets primero; desinstalar en orden inverso a las dependencias

Instancias Odoo

Síntoma	Causa probable	Solución
502 Bad Gateway	El contenedor no arrancó o el puerto no coincide	<code>docker ps</code> y <code>docker logs odoo-<instancia></code> ; comparar el puerto del compose con el del vhost
Aparece el selector de bases de datos	<code>dbfilter</code> o <code>list_db</code> mal configurados	Revisar <code>odoo.conf</code> y reiniciar el contenedor
Los enlaces de los correos llevan al dominio equivocado	<code>web.base.url</code> sin congelar	Fijar <code>web.base.url</code> y <code>web.base.url.freeze</code> (capítulo 11.4)
Las notificaciones en vivo no funcionan en la instancia del cliente	Falta el <code>location /websocket</code> en su vhost	Revisar la plantilla del vhost y el <code>gevent-port</code>
Error de certificado	El vhost 443 se desplegó antes de emitir el certificado	Seguir el orden del subplan SSL: HTTP primero, certificado, luego HTTPS
Lentitud generalizada en un servidor	Suma de <code>workers</code> de todas las instancias por encima de la CPU real	Revisar la densidad; bajar <code>odoo_workers</code> o repartir jets

16.3 Recuperar un jet atascado en estado transitorio

Si un flight plan muere de forma abrupta (reinicio de Odoo a mitad de ejecución, por ejemplo), el jet puede quedarse en `installing` o `starting` para siempre, y Tower rechazará toda acción nueva con `-505`.

1 Averigua qué pasó de verdad en el servidor

El estado en Odoo puede no reflejar la realidad. Comprueba con `docker ps` si los contenedores están arriba antes de decidir nada.

2 Alinea el estado del jet con la realidad

Desde **Ajustes > Técnico > Acciones de servidor**, o desde `odoo shell`:

```
jet = env["cx.tower.jet"].browse(ID)
estado = env["cx.tower.jet.state"].search([("reference", "=", "error")], limit=1)
jet.sudo().write({"state_id": estado.id, "target_state_id": False})
```

Escribir el estado directamente **no ejecuta ningún flight plan**: solo corrige el registro. Por eso hay que verificar antes el estado real del servidor.

3 Continúa por el camino normal

Con el jet en `error`, la acción **Recover** vuelve a ejecutar el plan de arranque y lo devuelve a `running`.

16.4 Qué vigilar en cada servidor

Métrica	Umbral	Comando de la ventana de log
Espacio en disco	> 80 %	<code>df -h /</code>
Memoria	> 85 %	<code>free -h</code>
Contenedores caídos	cualquiera	<code>docker ps -a --filter "status=exited"</code>
Consumo por contenedor	—	<code>docker stats --no-stream</code>
Caducidad de certificados	< 20 días	<code>certbot certificates</code>
Errores de nginx	—	<code>tail -100 /var/log/nginx/error.log</code>
Frescura de los backups	> 26 h	<code>ls -lt /opt/efactusv/backups head -20</code>

Configúralas como **Server Logs** de tipo *Command* en la plantilla de servidor (capítulo 8): así cada VPS nuevo nace con su panel de diagnóstico ya montado.

A. Mapa completo de menús

Ruta	Grupo mínimo	Contenido
Cetmix Tower	User	Menú raíz
Servers > Servers	User	Los servidores
Servers > Templates	Manager	Plantillas de servidor
Servers > Partners	User	Clientes con infraestructura
Jets > Jets	User	Las instancias
Jets > Templates	User	Plantillas de jet
Jets > Launch New Jet	Manager	Asistente de creación
Logs > Command Log	User	Historial de comandos
Logs > Flight Plan Log	User	Historial de planes
Logs > Jet Requests	User	Peticiones entre jets dependientes
Logs > Jet Template Installations	User	Instalaciones de plantillas en servidores
Commands > Commands	User	Catálogo de comandos
Commands > Flight Plans	User	Catálogo de planes
Files > Files	User	Archivos concretos
Files > Templates	User	Plantillas de archivo
Tools > Import YAML	—	Importación de configuración (módulo YAML)
Tools > Git Projects	—	Proyectos de código (módulo Git)
Settings > General Settings	Manager	Timeouts, notificaciones, crons
Settings > Variables > Variables	Manager	Definición de variables
Settings > Variables > Variable Values	Manager	Todos los valores, en una sola lista
Settings > Keys and Secrets	Manager	Llaves SSH y secretos
Settings > Jets > States	Manager	Catálogo de estados
Settings > Automation > Scheduled Tasks	Manager	Tareas periódicas
Settings > Automation > Shortcuts	Manager	Accesos directos a comandos
Settings > Automation > Webhooks	Root	Endpoints entrantes (módulo Webhook)
Settings > Automation > Webhook Authenticators	Root	Autenticadores (módulo Webhook)
Settings > Properties > Operating Systems	Manager	Catálogo de SO
Settings > Properties > Tags	Manager	Etiquetas

B. Referencia rápida de sintaxis

Sintaxis	Significado
<code>{{ mi_variable }}</code>	Variable de usuario, resuelta en cascada jet → servidor → global. Visible en los logs.
<code>{{ tower.server.ipv4 }}</code>	Variable de sistema, solo lectura
<code>#!cxtower.secret.mi_secreto!#</code>	Secreto. Enmascarado en los logs.

Sintaxis	Significado
<code>result = {"exit_code": 0, "message": "..."};</code>	Salida de un comando Python
<code>result = {"allowed": True};</code>	Salida de un autenticador de webhook
<code>custom_values["clave"] = valor</code>	Pasar datos entre comandos de un mismo flight plan

C. API de Python más usada

```
# --- Servidores ---
env["cx.tower.server.template"].create_server_from_template(
    ref_plantilla, nombre, ipv4=..., ssh_key=..., configuration_variables={...})
server.test_ssh_connection()
server.run_command(comando)
server.run_flight_plan(plan)
server.get_variable_value("mi_variable")

# --- Jets ---
jet_template.create_jet(server, name=..., partner_id=..., variable_values={...})
jet_template.install_on_servers(servers)
jet_template.uninstall_from_servers(servers)
jet.bring_to_state("stopped") # respeta permisos: usa este
jet.clone(server=otro_servidor, name="copia")
jet.get_variable_value("instance_domain")
jet.get_dependent_jets_by_template_reference("ref_plantilla")

# --- Archivos ---
file_template.create_file(server=server, if_file_exists="skip")
file.upload() # subir al servidor
file.download() # traer del servidor
file.delete() # borrar en el servidor

# --- Contratos (efactusv_tower_contract) ---
contrato.action_create_tower_jet()
contrato.tower_jet_ids
```

D. Lista de verificación: alta de un cliente nuevo

✓	Paso	Responsable
☐	Contrato creado y confirmado en Odoo	Comercial
☐	<i>Instance Template</i> e <i>Instance Server</i> asignados	Operaciones
☐	Servidor con capacidad por debajo del <i>Limit per server</i>	Operaciones
☐	Botón Create Instance pulsado; jet creado	Operaciones
☐	<code>instance_name</code> y <code>instance_domain</code> definidos y validados	Operaciones
☐	Puertos asignados sin colisión	Automático (10.7)
☐	Acción Create ejecutada; jet en <code>running</code>	Operaciones
☐	Registro DNS creado y propagado	Automático
☐	Certificado emitido; HTTPS válido en el navegador	Automático + verificación
☐	Base de datos inicializada con los módulos correctos	Automático
☐	Solo se ofrece la base del cliente (sin selector de BD)	Verificación
☐	Tarea de backup activa y con primera ejecución correcta	Verificación
☐	Ventana de log de la instancia visible para soporte N1	Operaciones
☐	Credenciales entregadas al cliente por canal seguro	Operaciones
☐	URL del jet rellena y enlazando a la instancia	Automático

E. Lista de verificación: baja de un cliente

✓	Paso	Cuándo
☐	Contrato terminado en Odoo	Día 0
☐	Instancia detenida automáticamente (verificar el chatter)	Día 0
☐	Volcado final de BD y filestore, guardado fuera del servidor	Día 0
☐	Entrega de datos al cliente si lo solicita	Días 1-30
☐	Periodo de retención cumplido	Día 30-90
☐	Acción Destroy ejecutada	Fin de retención
☐	Registro DNS eliminado	Automático
☐	Puertos liberados para reutilización	Verificación
☐	Certificado retirado de la renovación	Verificación

F. Glosario

Término	Definición
Access Level	Nivel (User/Manager/Root) que llevan tanto el usuario como cada registro. Un usuario solo ve registros de nivel igual o inferior al suyo.
Border action	Acción de creación (sin estado origen) o de destrucción (sin estado destino) de un jet template.
Command Log	Registro de una ejecución: código renderizado, salida, código de salida, duración.
Custom values	Diccionario efímero compartido entre los comandos de un mismo flight plan.
Flight Plan	Secuencia ordenada de comandos con lógica condicional.
Jet	Instancia de aplicación que vive en un servidor y tiene ciclo de vida propio. En efactusv, la instancia Odoo de un cliente.
Jet Action	Transición entre estados de un jet, ejecutada por un flight plan.
Jet Request	Petición interna generada cuando un jet necesita que otro alcance cierto estado.
Jet Template	Plano de un tipo de jet: estados, acciones, archivos, variables, dependencias, límites.
Post Run Action	Regla de una línea de plan que decide qué hacer según el código de salida.
Reference	Identificador técnico único y estable. Es la clave de emparejamiento en las importaciones YAML y en la API.
Secret	Valor sensible, referenciado con <code>#!cxtower.secret.x!#</code> y enmascarado en los logs.
Server Log	Ventana de observación configurada sobre un archivo o un comando del servidor.
Transit State	Estado que ocupa un jet mientras se ejecuta una transición. Bloquea acciones concurrentes.
Waypoint	Punto de restauración de un jet, con sus datos y sus variables.

G. Referencias

Recurso	URL
Documentación oficial de Cetmix Tower	https://cetmix.com/tower
Portal técnico de Tower	https://tower.cetmix.com
Comandos Python en Tower	https://cetmix.com/tower/documentation/command/#python-code-commands
API de OVHcloud, primeros pasos	https://docs.ovh.com/gb/en/api/first-steps/
Documentación de Boto3 (AWS)	https://boto3.amazonaws.com/v1/documentation/api/latest/index.html

Recurso	URL
OCA queue_job	https://github.com/OCA/queue
OCA contract	https://github.com/OCA/contract

Guía elaborada a partir del código fuente de los módulos del repositorio `efactusv-cetmix-tower` (rama 18.0), del módulo `efactusv_tower_contract` y del kit de arranque `tower/` del repositorio de despliegue. Las capturas proceden de la documentación incluida en los propios módulos y muestran datos de demostración.